

Programming Paradigms Introduction

Learning Outcomes

At the end of this discussion you should be able to:

1. Explain what a programming paradigm is.
2. Identify the four main programming paradigms
3. Explain why programming languages are shifting to multi-paradigmness

The title of this course

Let's start by talking about the title of the course name because it does sound like some buzz phrase. You will start saying this phrase soon so let's get the definition out of the way.

"Programming paradigms".

You're probably familiar what half of this phrase means, I mean, I hope you are, otherwise I don't know what to do.

Paradigm

Let's focus first on the non-obvious part, the word paradigm. This word comes up often in academia. You probably heard of the term **paradigm shift** somewhere, it describes some form of fundamental change in the way we think within scientific disciplines often characterizing a scientific revolution. One notable example of a paradigm is the shift from Ptolemaic or Geocentric cosmology to Copernican or Heliocentric cosmology.

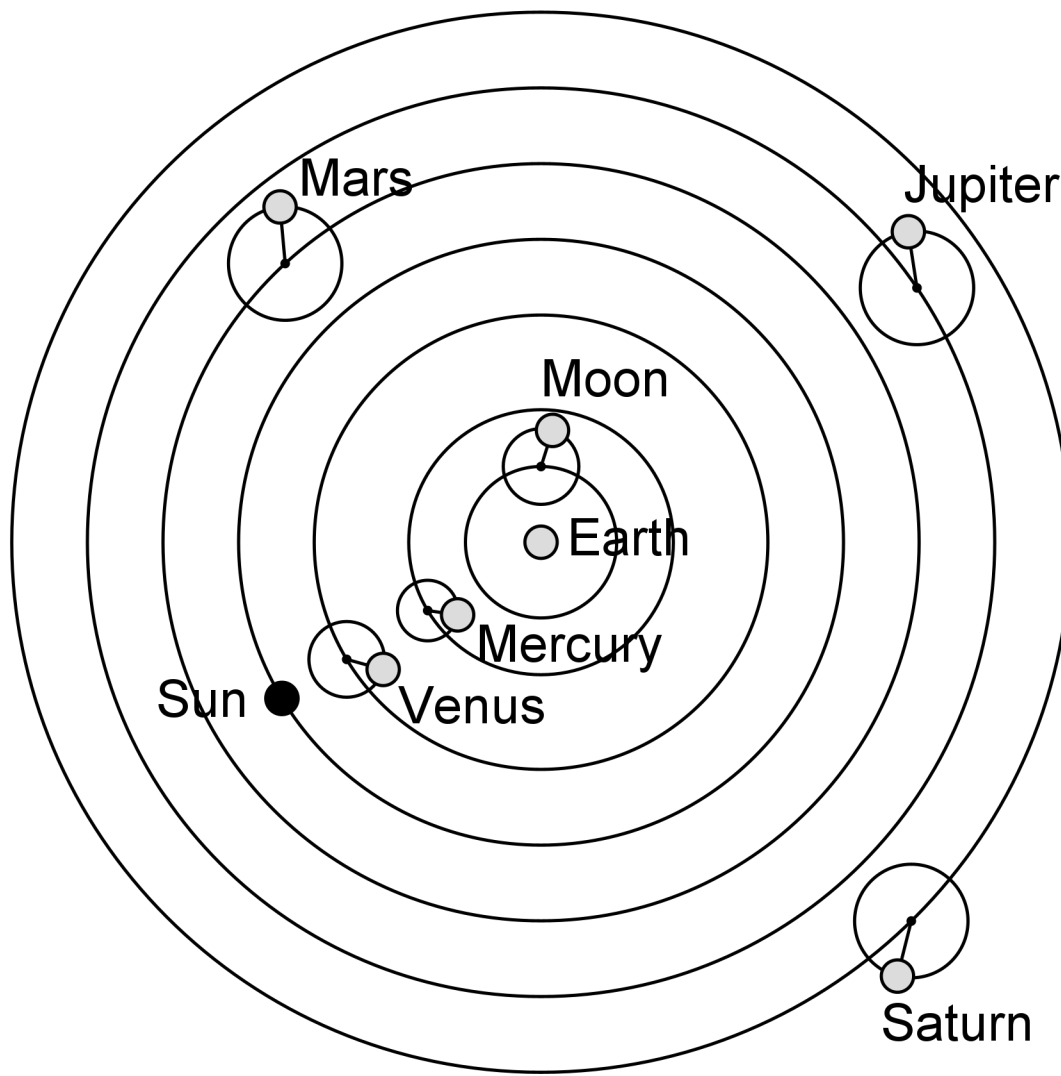


Figure 1: Ptolemaic Geocentric model of the solar system

Based on this context you can kind of formulate what the word paradigm means. This one connotes a similar meaning in the context of programming:

A paradigm is “*a framework containing the basic assumptions, ways of thinking, and methodology that are commonly accepted by members of a scientific community.*” (PARADIGM Definition & Meaning Dictionary.com, n.d.). It is a set of ideas and concepts that describe some **way of thinking**.

If we go back to the geocentric vs heliocentric paradigms in astronomy, you can't really definitively say that the heliocentric model is the only correct model of the solar system, you can still reconcile the geocentric model's perspective of putting the earth in the center by modelling heavenly bodies' movement with epicycles. After all, whether the earth or the sun is the center is a matter of perspective. It just so happens that placing the sun in the center provided science

with a more natural way of describing planetary movement. The heliocentric paradigm ended up uprooting geocentric paradigm as the dominant worldview, providing science with ideas that we still accept as truth until now, the earth is not the center. The earth is just one of the 9 planets, is not that special, gravity and inertia works in way which causes planets to move.

A paradigm shift like this is actually happening in programming language design, well talk about that some time later.

Programming

Now that you understand what paradigm means, let's talk about the first word, programming. As second year CS students, what is your definition of **programming**? It's a strange question to ask in a second year course, but I want you all to think about what the definition of programming is. The way you answer this question may actually tell you which perspective or programming **PARADIGM** you follow. When you say you are programming some kind of mechanism or behavior *what are you actually doing?* How do you define what a program is, and what is its relationship to a computer?

Here's one definition from the internet:

Computer programming is the process that professionals use to write code that instructs how a computer, application or software program performs. At its most basic, computer programming is a set of instructions to facilitate specific actions.(Cote 2025)

That is correct. Let me simplify that definition to this.

Programming is when you tell a computer what to do. When you write programs you're writing **instructions** for your computer.

1. Ask the user for a number
2. Store that number to a variable called x.
3. While x is greater than 7 do step 4 otherwise proceed
4. Subtract 7 to x and store the difference to x
5. Show the user the value of x

That is a good definition of programming. It gives you an understanding on how you write programs that work. All you need to do is to write correct instructions that the computer understands, and you'll have a perfectly working program. A **programming language** is a medium that describes how to write instructions to communicate to your computer. If you learn to do that then you can go ahead and program away.

It is a correct definition, but is it the *only* correct definition? It defines programming under the paradigm **imperative programming**. I will not begrudge you if this is the only definition you know since there is a huge likelihood that the only paradigm you've been exposed to has been imperative programming.

Taxonomy of Programming Paradigms

For someone who has been exposed to C, C++ and nothing else, you might feel that the natural way to code is the *imperative way* when in fact there are alternatives.

The diagram(MovGP0 2013) here represents the alternative schools of thoughts describing how to program. This diagram taxonomizes programming languages by identifying which paradigms they are under. Most of these paradigms are either not pragmatic, not popular enough or not unique enough to be studied in this course. Instead, we will be focusing on four major programming paradigms:

```
graph TD;
    Imperative-->Procedural;
    Imperative-->ObjectOriented;
    Declarative-->Functional;
    Declarative-->Logic;
```

Under the imperative family, procedural programming and objective oriented programming. Under the declarative family, functional programming and logic programming.

This course will give you an overview on these programming paradigms. Each of these are built upon the foundation of some mathematical formalism. We will explore the advantages and disadvantages of each paradigm while we take a tour through these four. Studying the disciplines upheld by these paradigms will also teach us good programming practices for designing elegant programs that transcend any programming paradigm.

Multi-paradigm programming languages

The way it used to be was that a programming language would be written with features adhering to the concepts of a particular paradigm. Sometimes, a language is written with fresh features that follow a different mathematical formalism that births its own programming paradigm. Back then paradigms worked like programming language **classifications**. The programming language C for example is a strong follower of procedural programming. Therefore, you can think of C as classified under procedural programming.

But as time passed by classifying a newer programming language under one paradigm became harder and harder. A programming language like python for example is mostly procedural, object-oriented, but sometimes functional.

Modern programming languages evolved to become **multi-paradigm**. This inevitably happened because, as programming languages age and grow, more **features** are added to it. These features are sometimes *borrowed from other paradigms* to solve a problem in a better way. This is the reason why established and mainstream programming languages like Java, C++, or Python tend to be multi-paradigm.

The multi-paradigmness of programming languages tend to be the reason why some programming language designers have abandoned the notion of building based on a strict paradigm. Instead, a language designer would *choose specific features* that they want to be supported on their programming language and implement it, regardless of its paradigm origins.

Imperative Programming Paradigm

Introduction

Imperative programming has turned out to be the *natural* paradigm of programming languages. The members of the imperative programming family have been dominating the market share of programming languages throughout the years with titans like BASIC, Pascal, C, Java and many more.

Learning Outcomes

At the end of this discussion you should be able to

1. Explain how imperative programming became the natural paradigm
2. Explain the concept of state in the context of imperative programming
3. Explain how the assignment statement enables the progression of states
4. Create structure programs to represent algorithms
5. Differentiate the subparadigms procedural programming and object-oriented programming

Quick Note on Imperative Programming and Procedural Programming

People usually use the terms Imperative programming and procedural programming interchangeably. Procedural programming is a subparadigm of imperative programming family, but some people refer to procedural programming as imperative programming. That's because other imperative paradigms like object-oriented programming is derived from procedural programming. You can think procedural programming as the ancestor if other imperative paradigms.

In this lecture I refer to Imperative paradigm as a whole, but I will focus on the main ideas that are common between other imperative paradigms. Ideas from object-oriented programming paradigm can be found on a separate lecture.

Popularity of the Imperative Paradigm

Imperative programming has turned out to be the *natural* paradigm of programming languages. The members of the imperative programming family have been dominating the market share of programming languages throughout the years with titans like BASIC, Pascal, C, Java and many more.

If you think about it, this is not that surprising. This paradigm's dominance could be attributed to most computer scientists' preference towards **pragmatic** and **efficient** programming languages. Especially since the most straightforward way of communicating to computer hardware is through the explicit manipulation of CPU memory and registers.

If you want the computer to do something for you, then you *communicate* to the computer that you want this and that to be done. And if you manage to give the computer correct and comprehensive **instructions** then you'll end up getting what you want.

If we rewind back to the dawn of programming languages you'll see that early programming languages were built to **communicate to computer hardware**. As a result of this, programming languages naturally adopted syntax with **imperative** moods.

Assembly programs for example was mostly built from sequences of executable instructions which was patterned from imperative statements from natural language.

```
INC ITER
MOV AH, 7
ADD AH, AL
```

For example the assembly instruction, `INC ITER`, tells the computer to increment the memory variable called `ITER`. The instruction `MOV AH, 7`, tells the computer to move the value 7 to the `AH` register. The instruction `ADD AH, AL` tells the computer to add the contents of the `AH` register to the `AL` register.

As time went by, newer *higher level programming languages* emerged (higher level meaning farther from hardware and closer to human language) like Basic, Pascal, and C. The syntax of these programming languages were written as **abstractions** of hardware code. Although programs written in these languages became more human-readable compared to its predecessors, these newer languages retained their imperative tones and mechanisms. This progression meant that higher level programming languages built atop of imperative languages naturally adopted the **imperative paradigm** as well. Java, Python, and C++ for example which were all written in C, followed this progression, thus establishing the imperative family as the dominant paradigm in programming language design.

The STATE

The existence of an explicit state is the foundation of imperative programming. The **state** of a program or a process on a given instance is the snapshot of its immediate relevant environment and context. The state of your CPU on a given instance for example will refer to the values found in the *registers and relevant memory*. On a specific process the state will refer to the values inside the *memory addresses* it resides in.

On a computer program the state can refer the conceptual set of variable values related to the program's runtime on some given instance. Let's use this program as an example:

```
int x = 3
int y = 4
x = x + y
```

At the start of runtime, the state of this program would be (*for all intents and purposes*) empty, since there are no relevant variables declared at this point. After executing the first line of code, the state of the program would look something like this:

variable	value
x	3

One integer variable named `x` with the value 3. After the next line of code a new variable is introduced and immediately assigned with the value 4 so the state of the program at this instance will look like this:

variable	value
x	3
y	4

And at the last line, the value of x is updated by adding the value of y, so the final state of this program will look like this:

variable	value
x	7
y	4

Assignment Statement

Another important construct of the imperative programming paradigm is the **assignment statement**. Assignment statements and the concept of state are very related to each other.

Assignment statements allow your program to *mutate* the values of your variables. Mutation in the context of programming is a fancy term that basically means change. And as we learned earlier, *changes to the context* of a program, which includes variables, creates states. Therefore, every **assignment statement**, corresponds to **new states** of a for the program.

Assignment statements are usually executed through the use of the “=” operator (some languages like Pascal use “:=” instead). Although it borrows the equality operator from math, assignment operators behave very differently from an equality statement. Instead of communicating some kind proposition, the assignment statement has an **imperative mood**. An equality $a=b$ in math **declares** that some a is b, while an assignment operator $a=b$ **commands** that a’s value is now the same as b. Mutation is introduced once you perform an assignment to a again, signifying a **change** in the value of a.

The closest corresponding mathematical construct to an assignment statement is the *let statement*. A statement in math such as “let x be equal to 3”, has an **imperative mood**. But unlike an assignment statement which can change the value of a variable any number of times, a let statement can only set the value of a variable once.

For every assignment statement you feed to the computer, something meaningful happens. That particular “something” is manifests as **changes** to your program’s context. The context of a program changes for every individual mutation of a variable. And you can compare the difference between the before and after of a specific assignment by comparing the **before-assignment** state and the **after-assignment state**. The *progression* from one state to another characterizes the effect of an assignment.

This is the important takeaway that you need to remember. Imperative programming is characterized by **imperative statements**. Statements that tell the computer what to do. The most important type of these statements is the **assignment statement**. An assignment statements effect to your computer is characterized by the progression from one **state** to another. *Assignment*

statements make states, and if you combine many of these assignment statements arranged in a particular manner, you can create a meaningful program that does something for you.

Structured Program Theorem

Creating meaningful programs in imperative programming is done by applying the **Bohm-Jacopini Theorem**, also known as the **Structured Program Theorem**. This theorem was one of the theoretical frameworks proposed to characterize imperative programming.

The theorem describes a formalism of a class called control flow graphs which are capable of representing any computable function. These control flow graphs are actually something you are intimately familiar of. It is known to you as the trusty old **flow chart**. Any control flow graph can be created by combining subprograms in three specific ways. A subprogram is a recursive unit of control flow graphs. A subprogram can be a single statement, or it can be a combination of more than one subprogram. Here are the three ways to combine subprograms:

1. Executing one subprogram, and then another subprogram (sequence)
2. Executing one of two subprograms according to the value of a boolean (selection)
3. Repeatedly executing a subprogram as long as a boolean expression is true (iteration)

The constructs described by this formalism became the natural architecture for programming language designers. This is the reason why CS students like you are introduced to programming using control flow graphs or flow charts. This is also the reason why programming languages like Pascal, C, Java and their derivatives are designed the way they are.

Subparadigms under the Imperative family

Procedural programming

Programming languages like Fortran, ALGOL, BASIC, and C fall under the **procedural paradigm**. Languages under this paradigm simplify a complex system by subdividing a program into different **procedures** or functions.

Object-oriented programming

Object-oriented programming focuses on modelling a system based on the real world ontology of **objects**. It uses an expressive type system to program the interactions within a system. Most modern programming languages, like C++, Python, Java, etc, have some object-oriented programming features.

Optional Readings

Rapaport W. (2004) Great Idea III: The Boehm-Jacopini Theorem and Structured Programming. CSE 111, Fall 2004 Accessed August 31, 2020

Functional Programming Formalism

During the 1930s a mathematician investigating the foundation of mathematics, named Alonzo Church, introduced a formal system of expressing computational logic. The system he created was called **Lambda Calculus**. It was until the 1960s when the system found its way through

different disciplines. It became something more than a mathematical formalism and became an important concept in linguistics and **computer science**(Church 1932).

Before we dive into functional programming let's introduce ourselves to the formalism that inspired it, Lambda Calculus. These concepts may seem strange at first since it imagines a mathematical foundation beyond numbers, sets, and logic.

Expressions in the Lambda Calculus Formalism

Let Λ be the set of expressions under the Lambda calculus formalism

1. **Variables.** If x is a variable, then $x \in \Lambda$
2. **Abstractions.** If x is a variable and $\mathcal{M} \in \Lambda$, then $(\lambda x.\mathcal{M}) \in \Lambda$.
3. **Applications.** If $\mathcal{M} \in \Lambda \wedge \mathcal{N} \in \Lambda$, then $(\mathcal{M}\mathcal{N}) \in \Lambda$.

Take a look at these important precedence conventions. You might get confused if you read some lambda calculus expressions. Some people often omit parentheses or single-parametrizations to write shorter expressions:

1. Application is left associative

$$\mathcal{M}_1\mathcal{M}_2\mathcal{M}_3 \equiv ((\mathcal{M}_1\mathcal{M}_2)\mathcal{M}_3)$$

2. Consecutive abstractions can be uncurried

$$\lambda xyz.\mathcal{M} \equiv \lambda x.\lambda y.\lambda z.\mathcal{M}$$

3. The body of an abstraction extends to the right

$$\lambda x.\mathcal{M}\mathcal{N} \equiv \lambda x.(\mathcal{M}\mathcal{N})$$

Reductions

Reductions are a ways to simplify and evaluate lambda expressions. You'll learn later that these reductions are basically concepts that are eventually adapted to functional programming concepts.

α equivalence:

α equivalence states that any bound variable, has no inherent meaning and can be replaced by another variable:

$$\lambda x.x \underset{\alpha}{=} \lambda y.y$$

Given a lambda calculus abstraction $\lambda x.\mathcal{M}$, this abstraction's bound variable is x . The bound variable x may appear somewhere in $\mathcal{M}$, the body of the abstraction. An alpha equivalence basically shows that the name of the variable has no inherent meaning. Therefore, you can replace it with any other variable name.

β Reductions

β reductions state how to simplify abstractions. This process is similar to applying a function in the context of programming. For example, we use the identity function $(\lambda x.x)$ and apply it to some free variable y .

$$(\lambda x.x)y \xrightarrow[\beta]{} y$$

When you beta-reduce some application $\mathcal{MN}$, what you're doing is replacing all instances of the bound variable in $\mathcal{M}$ with $\mathcal{N}$. Here's another example,

$$(\lambda u.\lambda v.uvu)\lambda x.x \xrightarrow[\beta]{} \lambda v.(\lambda x.x)v(\lambda x.x)$$

η reductions

η reductions describe equivalencies that arise because of free variables. If x is a variable and does not appear free in $\mathcal{M}$ then:

$$\lambda x.(\mathcal{M}x) \xrightarrow[\eta]{} \mathcal{M}$$

The lambda expression here is just some redundant abstraction. These η reductions characterize higher level simplifications that are not always as obvious as the other reductions.

Reduction example

For example to reduce the following lambda expression, we must first understand what it means.

$$(\lambda x.\lambda y.(xy))(\lambda x.\lambda y.(xy))$$

In the outermost level, the expression is the application of $\lambda x.\lambda y.(xy)$ to itself. It follows the second type of lambda calculus expression discussed earlier, $\mathcal{MN}$ where $\mathcal{M} \in \Lambda$ and $\mathcal{N} \in \Lambda$. In this context $\mathcal{M} = (\lambda x.\lambda y.(xy))$ and also $\mathcal{N} = (\lambda x.\lambda y.(xy))$.

When you start evaluating this expression, you might be tempted to automatically apply a β reduction by itself:

$$\begin{aligned} (\lambda x.\lambda y.(xy))(\lambda x.\lambda y.(xy)) &\xrightarrow[\beta]{} \lambda y.((\lambda x.\lambda y.(xy))y) \\ &\xrightarrow[\beta]{} \lambda y.\lambda y.(yy) \end{aligned}$$

But this reduction is actually incorrect because although x and y appear on both lambda expressions, these variables don't have the same meaning. The x and y variables inside the left lambda expression are **bound** inside this lambda expression. The x and y variables outside the left lambda expression (inside the right lambda expression) are **free** in its context, therefore, even though they look the same, it is incorrect to interchange the two variables.

To avoid confusion with similarly named variables, it is advisable to apply α equivalencies, to give variables different names. This can be done by replacing the right abstractions' bound

variables with u and v . Again, this alpha reduction doesn't change the meaning of the abstraction, it merely renames the bound variables.

$$(\lambda x.\lambda y.(xy))(\lambda x.\lambda y.(xy)) \equiv_{\alpha} (\lambda x.\lambda y.(xy))(\lambda u.\lambda v.(uv))$$

The correct reduction now is as follows. Still a β reduction but without the ambiguity of similar variable names.

$$\begin{aligned} (\lambda x.\lambda y.(xy))(\lambda u.\lambda v.(uv)) &\xrightarrow{\beta} \lambda y.((\lambda u.\lambda v.(uv))y) \\ &\xrightarrow{\beta} \lambda y.\lambda v.(yv) \end{aligned}$$

Lambda Calculus Encoding (Optional Reading)

One of the most interesting things that the Lambda Calculus System demonstrates is its use in computability theory. It's importance in computing has led to the formulation of the Church-Turing Thesis which conjectures that the Lambda Calculus System and the hypothetical Turing machine rules as complete representations of any algorithm. The lambda calculus system is said to be **Turing Complete**, which means that the system can simulate any conceivable Turing machine. To demonstrate its Turing completeness this topic will show some of the encodings in lambda calculus.

Boolean Values

A good starting point for encoding is the smallest unit of data, a boolean value. Boolean values, such as true and false can be represented by the following lambda expressions:

- **True:** $T = \lambda x.\lambda y.x$
- **False:** $F = \lambda x.\lambda y.y$

The way boolean values are encoded in lambda calculus using an abstraction that when applied to two values, produces first value for true and produces the second value for false. To demonstrate the consistency of this encoding we can use this encoding of an if_then_else function. An if_then_else function in the context of any familiar programming language looks like this:

```
if(condition)
  this
else
  that
```

The function consumes three expressions (condition, this, and that) If the condition is true, then the function produces this, otherwise the function produces that. This function can be represented in lambda calculus as the following function:

$$\lambda c.\lambda x.\lambda y.cxy$$

$c : \text{condition}, x : \text{this}, y : \text{that}$

Applying this function shows how lambda calculus boolean encodings work. The example below shows what happens when the condition is applied to a true condition

$$\begin{aligned}
 (\lambda c.\lambda x.\lambda y.cxy)Tab &= (\lambda c.\lambda x.\lambda y.cxy)(\lambda u.\lambda v.u)ab \\
 &= (\lambda x.\lambda y.((\lambda u.\lambda v.u)xy))ab \\
 &= (\lambda x.\lambda y.((\lambda v.x)y))ab \\
 &= (\lambda x.\lambda y.x)ab \\
 &= (\lambda y.a)b \\
 &= a
 \end{aligned}$$

1

The example below in the other hand is the same if_then_else function applied to a false condition

$$\begin{aligned}
 (\lambda c.\lambda x.\lambda y.cxy)Fab &= (\lambda c.\lambda x.\lambda y.cxy)(\lambda u.\lambda v.v)ab \\
 &= (\lambda u.\lambda v.v)ab \\
 &= b
 \end{aligned}$$

2

By representing the if_then_else function in lambda calculus we are also able to encode all the logic gates by reusing our if_then_else function.

not function

The not function can be represented using the if_then_else function this way:

```

if(condition)
  False
else
  True

```

In lambda calculus:

$$\lambda x.xFT$$

or Function

The or function can be represented using the if_then_else function this way:

```

if(left)
  True
else
  right

```

In lambda calculus:

$$\lambda x.\lambda y.xTy$$

¹In the first line a quick α equivalency is done on the True encoding.

²In the effort of saving lines β reductions with currying are applied in one go.

and Function

The or function can be represented using the `if_then_else` function this way:

```
if(left)
  right
else
  false
```

In lambda calculus:

$$\lambda x.\lambda y.xyF$$

Natural Numbers

Natural numbers are encoded in lambda calculus similar to how numbers are described using *Peano's Axioms*. By defining 0 and defining a successor function. This encoding is called Church numerals. The first natural number 0 is defined as the lambda expression:

$$\bar{0} = \lambda s.\lambda z.z$$

Which is α equivalent to the encoding for boolean false.

All other natural numbers are derived using a special function called the successor function, such that the successor of any natural number n is $n + 1$. The successor function in lambda calculus is represented by:

$$S = \lambda n.\lambda s.\lambda z.(s(nsz))$$

Using this function we can derive the encoding for the natural number $\bar{1}$:

$$\begin{aligned}\bar{1} &= S(\bar{0}) = (\lambda n.\lambda s.\lambda z.(s(nsz)))(\lambda u.\lambda v.v) \\ &= \lambda s.\lambda z.(s((\lambda u.\lambda v.v)sz)) \\ &= \lambda s.\lambda z.(s(z))\end{aligned}$$

To derive this encoding for two we simply find $S(\bar{1})$.

$$\begin{aligned}\bar{2} &= S(\bar{1}) = (\lambda n.\lambda s.\lambda z.(s(nsz)))(\lambda u.\lambda v.(u(v))) \\ &= \lambda s.\lambda z.(s((\lambda u.\lambda v.(u(v)))sz)) \\ &= \lambda s.\lambda z.(s(s(z)))\end{aligned}$$

Continuing this process and generalizing successions will lead us the following encoding of natural numbers:

Church Numeral	Lambda Calculus Encoding
$\bar{0}$	$\lambda s.\lambda z.z$
$\bar{1}$	$\lambda s.\lambda z.(s(z))$
$\bar{2}$	$\lambda s.\lambda z.(s(s(z)))$
$\vdots$	$\vdots$
$\bar{n}$	$\lambda s.\lambda z.s^n(z)^3$

Another useful definition of a successor function would be:

$$S(\bar{n}) = S(\lambda s.\lambda z.s^n(z)) = \lambda s.\lambda z.s(s^n(z)) = \lambda s.\lambda z.s^{n+1}(z)$$

Addition and Multiplication

The successor function will help us derive the lambda calculus representation of an addition function. In the same sense that addition is just a repetition of increments, we can implement addition by making use of repetitive applications of the successor function.

$$\text{add} = \lambda m.\lambda n.mSn$$

To test the consistency of this function, we can try adding two arbitrary Church numerals $\bar{m}$, and $\bar{n}$.

$$\begin{aligned}
\text{add } \bar{m}\bar{n} &= (\lambda m.\lambda n.mSn)\bar{m}\bar{n} \\
&= \bar{m}S\bar{n} \\
&= (\lambda s.\lambda z.s^m(z))S(\lambda a.\lambda b.a^n(b)) \\
&= (\lambda z.S^m(z))(\lambda a.\lambda b.a^n(b)) \\
&= S^m(\lambda a.\lambda b.a^n(b)) \\
&= \lambda a.\lambda b.a^{m+n}(b) \\
&= \lambda s.\lambda z.s^{m+n}(z) \\
&= \overline{m+n}
\end{aligned}$$

Multiplication works similarly. To solve for the product of two Church numerals $\bar{m}$ and $\bar{n}$, we simply add n to 0 repetitively, m number of times. The function will look similar to a `add`, but with a partial application of `add n` instead of the successor function `S`.

$$\text{mult} = \lambda m.\lambda n.m(\text{add } n)\bar{0}$$

³ $\mathcal{M}^n(\mathcal{N})$ is a shorthand notation for n applications of $\mathcal{M}$ to $\mathcal{N}$ or $\mathcal{M}(\dots(\mathcal{M}(\mathcal{M}(\mathcal{N})))\dots)$.

Functional Programming Basics

Reimagined functions

Lambda calculus evolved from a system of logic foundation with deep roots to computation theory into something that became a basis for *programming language design*. Language designers started to consider the unconventional representation of lambda calculus expression as a valid and pragmatic way of *representing data*. Around 1950s programming languages patterned around the framework of lambda calculus started to emerge. One of the earliest and most important of these languages was **Lisp** which evolved to become a large family of programming languages (Steele and Gabriel 1996). Soon, more programming languages started to implement the same formalism described by lambda calculus. This opened a new paradigm of programming languages called **functional programming paradigm**. To explore this paradigm this section will introduce the programming language **Haskell**. This language has become one of the most important functional programming language, setting the standards for other languages' paradigm.

How functions are treated Differently

One of the biggest difference between your classic imperative programming languages like C and Java and a functional programming language, is how it treats its *functions*. You might have probably guessed that since the paradigm has “function” in its name. To explore this contrast lets start by introducing a simple function, written in C. This function basically accepts an integer and returns the same integer but squared:

```
int square(int x){  
    return x*x;  
}
```

Functions like these are patterned from mathematical functions. It has a **name** to invoke it later called square, it has **specifications** on which type of data it accepts (int x) and produces (int), and finally it has **instructions** on what must be done when it is invoked (return x*x). Functional programming functions behave in more or less the same way.

```
square :: Int -> Int  
square x = x * x
```

It looks different but all the parts you can find in a C function can also be found on this Haskell function.

In terms of invocation, they are also used similarly and of course they behave similarly:

```
square(5);  
square 5
```

Although functions in non-functional programming behave and look similar to functions in functional programming language, they have a huge difference in the way the programming language *treats* it.

A function in C is treated differently from other types of data. In fact C programmers will rarely call a function a *value*. What this means is that canonical value types like integers, characters,

and arrays (even compound value like struct instances and objects) can be *passed* on functions and can be *returned* as functions.

```
int* add_to_array(int arr*,int x,int size){
    for(int i=0;i<size;i++){
        arr[i]+=x;
    }
    return arr;
}
```

C discriminates function from these canonical value types. Therefore, during runtime, non-functional programming languages interpret the expression `square(5)` as “the number 5 squared” while the expression `square` is just some disembodied function name (*square of which number?*). Imperative programming functions during runtime are meaningless unless they are directly invoked.

Higher Order Functions

Passing functions

Functional programming languages treat functions the same way it treats values, you *can* pass them in other functions, and you *can* return them as well.

```
s:: Int -> Int
func x = x + 1

p::Int -> Int
func x = x - 1

applytwice:: (Int-> Int) -> Int -> Int
applytwice f x = f (f x)
```

The code above shows two function definitions with some *type signature annotations* for readability. The first is the function `s::Int -> Int` which is *applied to an integer* and *produces an integer*. What it does is it simply adds one to x. The second function is similar but what it does is subtracting one from x.

Type signature annotations are not required here, that’s why they’re called *annotations*. Adding these annotations will *restrict* the type the functions can be applied to. Type signature annotation syntax are understood like this

```
f :: Paramtype -> AnotherParamtype -> ... -> OutputType
```

The last type in the `->` series is the type the function produces (similar to its return type) and everything else before it are parameter types.

The third function called `apply_twice` is what we call a higher order function. A **higher order function** is a function that either accepts a function as a parameter or returns a function parameter or both. The function `applytwice`, as described by its *type signature*, is applied to a function `f` and an integer `x` and produces an integer. It applies the function `f` twice to `x`, something like $f(f(x))$.

By defining a function like this we can do something like this during runtime:

```
ghci> applytwice s 3
5
ghci> applytwice p 3
1
```

To a programmer with no experience with functional programming, this feature can be surprising especially since mathematical functions in algebra or calculus don't even explore this capability. But if you remember this is not just some arbitrary added feature added for novelty. This feature is directly patterned from lambda calculus:

$$\begin{aligned}\text{let } S &= \lambda n. \lambda s. \lambda z. (s(nsz)) \\ T &= \lambda f. \lambda x. (f(fx)) \\ \bar{5} &= TS\bar{3}\end{aligned}$$

In lambda calculus an *abstraction* and an *application* does not restrict anyone from the type of expressions bound to variables. In the spirit of implementing lambda calculus, any functional programming language will allow you to do this as well.

Returning functions

On the other side of the coin, a function, in functional programming will also let you *return functions* the same way you *return* any other kind of data.

To explore this, suppose we have different functions that when applied to an integer, produces that integer plus a certain integer.

```
addTwo :: Int -> Int
addTwo x = x + 2

addThree :: Int -> Int
addThree x = x + 3

addFour :: Int -> Int
addFour x = x + 4
```

We can generalize these functions into a *function-maker* function, that when applied to an arbitrary integer x, will produce a function similar to addx which is a function that adds x to your integer.

```
addMaker :: Int -> (Int->Int)
addMaker x = (\y -> x + y)
```

We are introducing new syntax here, but this new syntax is a representation of an expression we already know from lambda calculus. The definition for your addMaker ($\backslash y \rightarrow x + y$) is basically an implementation of the following *lambda expression*⁴. y is the bound variable, and the operator

⁴In fact, the reason why Haskell syntax uses the $\backslash$ character to represent lambda expressions is because this is your keyboard's best physical approximation of the Greek letter λ .

-> separates the inputs and the output, $x + y^5$.

$\lambda y. \text{add } xy$

What this expression means then is that `addMaker` produces a *lambda expression*, which essentially behaves exactly like a function. This allows you to create functions during runtime.

```
ghci> addSix = addMaker 6
```

Simply writing the expression `addSix` on your terminal will yield you an *error*, because printing `addSix` doesn't really have a meaning outside the world of lambda calculus. It is a lambda expression which is *basically* a function. *How do you represent a function as a string?*

But since `addSix` is a lambda that behaves exactly like a function, you can apply `addSix` to an integer, and it will give you a meaningful answer.

```
ghci> addSix 3
9
```

In fact, you can even omit the part where you bind the value returned by `addMaker` to a name, and instead use it directly. Here, `(addMaker 7)` is a lambda expression, therefore it can be applied to an integer.

```
ghci> (addMaker 7) 4
11
```

This nifty trick right here is the reason why lambda expressions are also called **anonymous functions** since these expressions on their own don't have a name. Lambda expressions generally appear in functional programming languages and even non-strictly functional programming languages. Lambdas can be useful if you want to create a function that will be used only once:

```
ghci> applytwice (\x -> x + 2) 3
7
ghci> (\x -> x * x) 4
16
```

Lambdas, just like any other canonical value type can be bound to identifiers⁶. Doing this will **name** the lambda thus allowing it to behave just like any other named function.

Closure

A higher order function like `addMaker` above, is not *only* producing the lambda inside its definition. What is actually being produced is a construct called a **closure** which is the function definition described by the *lambda* and the *environment* of the function call. The extra data, called environment, is the reason why the lambda `(\y -> x + y)` makes sense outside the context of

⁵Extra note: “+” does not exist in the universe of lambda calculus so instead what's used here is a reference to a lambda calculus abstraction called “add”.

⁶Bindings are haskell's representation of a mathematical “let” statement. When you see a = operator like `x = 3`, this is not an assignment statement, but instead a let binding. In this case `3` is bound to the identifier `x`. Similar to what happens when you say let $x = 3$ in math.

`addMaker`. Without passing the *environment*, the variable `x` would be a free variable which will yield you a compilation error.

Inside your `addmaker` when you evaluate `addMaker 6`, the parameter `6` is *bound* to the variable `x`. Therefore, the resulting lambda produced by `addMaker 6` will behave exactly like the lambda, $(\lambda y \rightarrow 6 + y)$.

Closures are still a direct consequence of lambda calculus' variable binding rules. Specifically how the variables `x` and `y` are bound in the innermost body of the lambda calculus abstraction $\lambda x. \lambda y. add\ x$.

Currying

If we look back to lambda calculus you'll notice how abstractions are defined to be:

$$\lambda x. \mathcal{M}$$

Here we can see that abstractions are defined to have exactly one parameter. One can argue that this is different from how functional programming represents its own functions and lambdas since functions with *multiple parameters* are allowed in these languages. As it turns out, these functions are just *disguised* to have multiple parameters. These functions are just several single parameter functions combined to *simulate* multiple parameter functions. As an example: a function `add` that adds two numbers may look like multiple parameter functions:

```
plus x y = x + y
```

Internally, this function is equivalent to two lambda calculus abstractions, nested together to simulate *multiparameterness*.

```
plus = \x -> (\y -> x + y)
```

Here `plus` is a higher level function that accepts a single argument `x` and produces the closure $(\lambda y \rightarrow x + y)$. This expression is a direct implementation of the following lambda calculus abstraction:

$$plus = \lambda x. \lambda y. add\ xy$$

Just like lambda calculus, Haskell's `->` operator is right associative, so you can write the same *plus* function as:

```
plus = \x -> \y -> x + y
```

You can even omit the first `->` and the `\` near `y`, and it will mean the same lambda expression:

```
plus = \x y -> x + y
```

Which looks almost exactly similar to a relaxed lambda calculus expression with multiple parameters:

$$\lambda xy. add\ xy$$

Now when you want to apply this function we write:

```
ghci> (plus 3) 4
7
```

Which means that: first, we are evaluating `(plus 3)` which will give us a *closure*. The closure is then *applied* to 4 which completes the evaluation to 7. This is also a direct implementation of a lambda calculus application:

$$(\text{plus } \overline{3})\overline{4}$$

And just like lambda calculus, function applications in Haskell are also left associative, so you can omit the parentheses:

```
ghci> plus 3 4
7
```

All multiple parameter functions and lambdas in Haskell are nested single parameter abstractions in disguise, so for all intents and purposes, these two definitions for `plus` behave in the exact same way:

```
plus x y = x + y
```

```
plus = \x -> (\y -> x + y)
```

This means that the expression `(plus 3)` will have the same meaning regardless of the way you define `plus`. The expression `(plus 3)` has a special name, it is called a **partial application**. When you apply a function that is supposed to accept n parameters to m values (where $m < n$), this means is that you're supplying the function less parameters than it is expecting. Instead of getting the value, you get a partial application of that function which will evaluate to a *closure*.

The process of converting a multi parameter function or lambda to a nested single parameter lambda is called **currying**. This term is named after the mathematician Haskell Brooks Curry, which is the same Haskell, the programming language is named after.

A Stateless Paradigm

Functional purity and the absence of states

One of the most distinguishing aspects of functional programming and the declarative family of languages is its philosophy of statelessness. A programmer primarily exposed to mutable imperative programming languages will find that the concept of state is natural and maybe even inevitable. Functional paradigm challenges this concept and offers a much safer and mathematically intuitive philosophy. In the perspective of functional programming, there is no state, and everything is immutable.

Pure functions

To understand the functional programming perspective, we have to take a step away from the imperative programming definition of a function. Let's go back to the definition of a function in mathematics.

Across several, mathematical disciplines a function means the same thing. Consider two sets A and B . We can define a function as the *mapping* between the elements of A and B . The elements of A and B can be anything, they can be numbers, which shows us how a function can be represented by a formula or a graph. The elements of A and B can be matrices and vectors, which defines a function as a transformation between two vector spaces. On the higher level perspective of category theory, functions are *morphisms* between objects of a given category.

$$f : (A \rightarrow B)$$

If you remember functions from discrete math, functions at its most basic form looks like the one above.

Functions in functional programming languages like Haskell are (arguably) the closest computer representation of a mathematical function. We call these functions, **pure functions**.

They differ from your standard C function because the definition inside pure functions are only instructions on how to produce a result based on the parameters. To fully understand this concept here are some examples of impure functions

```
int square(int x){
    addToExternalLogger("calculating square");
    return x*x;
}
```

The impurity in this function is the line where the function writes to some external logger, `addToExternalLogger("calculating square");`. A function can only be pure if the result of the function can be fully determined by its parameter. The only parameter here is x . Invoking this square function with the same parameter value does not do the same thing. The effect of changing the logger is dependent on the previous state of the external logger. The effect on the logger is what we call a **side effect** of the square function. It is a side effect since this line of code modifies values outside boundaries of the function.

```
int* increaseArray(int *a, int size){
    for(int i = 0; i < size; i++)
        a[i]+1;
}
```

A pass by address/reference function which changes the value of a parameter will automatically be an impure function since changing the value of a is a **mutation**, which is a side effect.

```
String headsortails(){
    if(randInt()%2==0)
        return "heads";
    else
        return "tails";
}
```

This function is also impure because the return value is not dependent on the parameters alone. The return value will be dependent on the randomization seed which is something outside the parameters of the function.

A pure function must satisfy these two:

1. A pure function has no side effects
2. A pure functions output must be dependent on the inputs alone⁷

A good way to test if a function is pure is if you can (theoretically) create an infinitely long *lookup table* such that, looking up the value for a specific input is perfectly identical to calling the function with the same input. And if you think about it this is the essence of a function. Functions are just of mappings between the domain and the range,

The Absence of mutation

One of the hallmarks that make imperative programming imperative is the assignment statement. It enables the program to advance to a new state. Purely functional programming languages like Haskell, lacks the mechanism to mutate anything. Using the “=” operator (which signals an assignment statement in imperative languages) in functional languages *binds* the value on the right-hand side to the left-hand side. This mechanism is conceptually different from an assignment operation in C. It is perfectly fine to do the following in C:

```
int x = 0;
x = 1;
```

This code in C starts with a combined declaration and assignment: `int x = 0`. The second line, **reassigns** the same variable `x` to the new value 1. These lines of code correspond to a *mutation* on the variable `x`, (from 0 to 1). Because mutation can happen anytime during runtime, the value of the variable `x` is not definite. This is why values of variables in imperative programming **depend on the current state** of the program.

On the other hand, replicating this C code in Haskell is in fact not allowed:

```
x = 0
x = 1
```

It will give an error message upon compilation:

```
main.hs:2:1: error:
  Multiple declarations of 'x'
  Declared at: main.hs:1:1
             main.hs:2:1
```

In Haskell, any “=” statement is a *declaration of a binding*. These bindings are *final* (in the scope of the identifier). It is even wrong to call `x` here a variable since its value does not vary. The correct way to call `x` is *identifier*, since it merely identifies the value bound to it.

Consequences of Statelessness and Immutability

Haskell’s functions are better representations of mathematical functions. But what is the point of faithfully representing mathematical functions?

⁷In fact if $f(a) = b$ and $f(a) = c$ where $b \neq c$, then f is not a function at all

There are several reasons why programming without state can lead to better code. Eliminating all side effects is demonstrably *safer* against accidental errors. Building a library of functions perfectly working without worrying about side effects makes the system easier to understand and more resilient to changes.

```
void f(int *x, int y){
    *x = *x + y;
    printf("%d\n", *x);
    return;
}
int main(void) {
    int x = 0;
    f(&x, 3);
    x = x - 2;
    f(&x, 3);
}
```

The exact behavior of the function `f()` **depends on where you use it**. Even if you use the same parameters, you're not guaranteed to get the same results.

On small chunks of code, managing the consequences of having states such as global variables, will be trivial since you can reasonably track which variables are global (*or external in general*) and which functions interact with the global variables. Even with a few lines of code like the example, the function's effects and *side effects* are not very obvious.

As the system grows, using functions and variables without double-checking for side effects becomes much harder. As a consequence the whole system becomes a nightmare of impure functions on top of impure functions which may unexpectedly affect other parts of the system. On a corporate setting where multiple people are working on the same system, refactoring becomes *unsafe* without knowledge of all the side effects of the functions in use. On systems with *shared resources* and multi-threading it becomes extra-extra difficult to keep track of things without proper documentation.

Even with all these disadvantages in the state-full mutable paradigm of imperative programming, *one can still write robust and harmonious code*. You just have to be extra careful writing your code with discipline, *only using global variables and side effects when it is safe and necessary*. This is in fact the reason why *writing smaller pure functions* is considered good practice in any paradigm. After all, being able to code with states can be thought of as an extra feature. You can be a C programmer and just treat all your variables as immutable and all of your functions as pure.

Advantages and Disadvantages of Functional Programming

Advantages

Functional programming's main advantage lies in its stateless paradigm. Without state, you can ensure that your programs are easy to write without worrying about side effects. You can build up complex programs by writing small pure functions and compose them with each to model any algorithm.

Higher-order functions allow functions be very flexible. Combined with partial application, even simple functions can be surprisingly powerful.

Disadvantages

Functional programming has its own set of *disadvantages* as well, most of them related to this seemingly artificial crutch of statelessness and immutability. The most obvious one is that creating new values instead of changing an existing variable has extra overhead in both processing and memory, making functional programming *slower and less efficient*. Programming without state can be difficult to do for certain mechanisms (*Like the external logger for example*). Simulating mechanisms like this may introduce conflict on how you compose your functions. It's not impossible though, you just have to learn some category theory concepts such as **monads**. Also, for most people who are used imperative programming, *recursion*, does not feel natural compared to *iteration*. But in my opinion, after being exposed to functional programming for some time, recursion can make more sense than iteration.

Nevertheless, these disadvantages are not insurmountable. In fact, there are a plenty of systems written in functional programming languages running without issues. Functional programming has had the reputation of being more conceptual and fancy than the classic imperative programming language. People mocked Haskell for its pristine white tower “elitist” approach to programming. But recently these functional programming languages like Haskell, F# and Scala has enjoyed improvements that have elevated them to be as pragmatic as your classic C, C++ or Java. In fact, functional programming has gained a considerable rise in popularity to an extent that mainstream programming languages with imperative roots like C#, Python, and JavaScript have started to introduce *features patterned from pure functional programming languages*. Features such as **higher-order functions** and **lambdas**. With the risk of sounding editorial I even argue that learning functional programming concepts has become a *necessity* for any programmer, regardless of his/her paradigm preferences.

Logic Programming Formalism

Logic programming is based on the formal system of **predicate calculus**. It is a formalism based on *logical operations* and *quantification*.

Satisfiability and Horn Clauses

Logic programming applies the formalism of predicate calculus as means to prove statements and look for logical resolutions. To introduce these concepts, we will first talk about the satisfiability problem and horn clauses.

Satisfiability

A boolean expression is said to be **satisfiable** if there exists an assignment of truth values (either TRUE or FALSE) that evaluates the entire formula to TRUE. A **boolean expression**, is an expression that uses only boolean values, variables or boolean operations.

Here's a trivial example of a satisfiable boolean expression:

$$p \vee q \vee r$$

This expression will evaluate to true if p , q , and r are all set to TRUE.

Here's a trivial example of an unsatisfiable expression:

$$p \wedge \neg p \wedge q$$

It is not possible for this expression to evaluate to TRUE, since $p \wedge \neg p$ is a contradiction (it always evaluates to false).

These examples are trivial since, the expressions are short enough, or there is an obvious contradiction that resolves the formula. But in general, satisfiability is one of the hardest problems in Computer Science. There are no fast algorithms that can find solutions to a general satisfiability problem.

For example try to check if the following formula is satisfiable:

$$\begin{aligned} &(u \vee \neg v \vee w) \wedge \\ &(\neg u \vee v \vee p) \wedge \\ &(\neg u \vee p \vee r) \end{aligned}$$

To make it a little easier to solve satisfiability problems, we usually write boolean formulas in **conjunctive normal form** (clausal normal form or CNF). A boolean formula is in CNF if it is a *conjunction of disjunctions of boolean literals*. Each disjunction is also known as a clause (e.g. $(u \vee \neg w \vee w)$). We call unnegated values like u as a positive literal while negated values like $\neg w$ as a negative literal.

Any boolean formula can be expanded into CNF using **logical equivalences**. The equivalent CNF of a boolean formula is easier to solve since we just need to find satisfiable assignments for each clause consistently (which is still not difficult).

Horn Clauses

Horn clauses are special clauses where there is at most one positive literal in the disjunction. A boolean formula in CNF where all clauses are Horn clauses is called a **Horn formula**. The satisfiability of a Horn formula is much easier to solve.

$$\begin{aligned} &(\neg p) \wedge \\ &(\neg q \vee r) \wedge \\ &(\neg r \vee \neg s \vee t) \wedge \\ &(\neg s \vee \neg t) \end{aligned}$$

If each Horn clause in the formula has at least one negative literal, then the formula is *always satisfiable* by assigning FALSE to each variable. This is because the negative literals will all evaluate to TRUE, therefore making each clause TRUE. This Horn formula is a trivial case for satisfiability.

On the other hand if some of the clauses have no negative literals, then we can simply reduce the clauses to either a trivial satisfiable Horn clause or a contradiction.

$$\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(s) \wedge \\
&(\neg r \vee \neg s \vee t) \wedge \\
&(\neg s \vee t)
\end{aligned}$$

In this example, one of the Horn clauses have no negative literal. Note, that a Horn clause with no literal is a clause with exactly one positive literal.

This can be solved by assigning clause (s) as TRUE, since it can be true for the entire formula to be satisfiable.

$$\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\top) \wedge \\
&(\neg r \vee \perp \vee t) \wedge \\
&(\perp \vee t)
\end{aligned}$$

In the previous formula, TRUE is denoted by the $\top$ symbol, and FALSE is denoted by the $\perp$ symbol.

We just need to reduce the formula according to logical equivalencies:

$$\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\neg r \vee t) \wedge \\
&(t)
\end{aligned}$$

After reducing, one of the Horn clauses, (t) , has no negative literal. Which means it must be assigned TRUE.

$$\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge \\
&(\neg r \vee \top) \wedge \\
&(\top)
\end{aligned}$$

Reduced into:

$$\begin{aligned}
&(\neg p) \wedge \\
&(\neg q \vee r) \wedge
\end{aligned}$$

This formula is a trivial Horn formula (all clauses have at least one negative literal). The remaining clauses will be assigned FALSE, while s and t will be assigned TRUE. This means that this Horn formula is satisfiable.

In this other example, the Horn formula can be reduced into a contradiction, which means that it is unsatisfiable.

$$\begin{aligned} &(\neg p) \wedge \\ &(\neg q \vee r) \wedge \\ &(s) \wedge \\ &(t) \wedge \\ &(\neg s \vee \neg t) \end{aligned}$$

First, we assign TRUE to s .

$$\begin{aligned} &(\neg p) \wedge \\ &(\neg q \vee r) \wedge \\ &(\top) \wedge \\ &(t) \wedge \\ &(\perp \vee \neg t) \end{aligned}$$

Apply logical equivalencies to reduce.

$$\begin{aligned} &(\neg p) \wedge \\ &(\neg q \vee r) \wedge \\ &(t) \wedge \\ &(\neg t) \end{aligned}$$

Assign TRUE to t .

$$\begin{aligned} &(\neg p) \wedge \\ &(\neg q \vee r) \wedge \\ &(\top) \wedge \\ &(\perp) \end{aligned}$$

In this case, we end up with a FALSE clause, which means that the conjunction cannot be true. Therefore, this Horn formula is not satisfiable.

Horn Clauses as Implications

Horn clauses also have the advantage of being neatly rewritten as *implication statements*. If you apply the logical equivalency that converts disjunctions into implications. You can convert your Horn clauses into implication statements.

$$\neg p \vee q \equiv p \rightarrow q$$

A Horn clause that has exactly one positive literal can be converted into an implication with the sole positive literal as the conclusion, and the rest as the hypothesis.

$$\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_n \vee q \equiv \\ (p_1 \wedge p_2 \wedge \dots \wedge p_n) \rightarrow q$$

A Horn clause with exactly one positive literal and one or more negative literal is known as a **definite Horn clause**.

A Horn clause that is only made up of one positive literal and nothing else is called a **fact**. To convert it into an implication, we first write it as a disjunction using the identity property of disjunctions.

$$q \equiv \perp \vee q \\ \equiv \top \rightarrow q$$

Note that, a Horn formula made up of only facts and definite Horn clauses is always satisfiable by assigning TRUE to every variable. By assigning, TRUE to every variable, you ensure that each positive literal is TRUE, meaning each Horn clause is also TRUE.

A Horn clause that is made up 1 or more negative literals, is called a **goal clause**.

$$\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_n \equiv \\ \neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_n \vee \perp \equiv \\ (p_1 \wedge p_2 \wedge \dots \wedge p_n) \rightarrow \perp$$

Resolution

Resolution is one of the inference rules of logic. According to resolution, assuming two clauses with complementary literals (a pair of literals that are negations of each other), you can conclude the resolvent. The resolvent is a disjunction of all the non-complementary literals from both clauses.

$$p_1 \vee p_2 \vee \dots \vee r \\ q_1 \vee q_2 \vee \dots \vee \neg r \\ p_1 \vee p_2 \vee q_1 \vee q_2 \dots$$

In this example, the complements r and $\neg r$ are cancelled out from the resolvent.

Horn clauses are closed over resolution. This means that the resolution of two Horn clauses is guaranteed to be a horn clause. This property ensures that conclusions inferred from Horn clauses will always be Horn clauses. Because of this, it will always be easy to check the satisfiability of Horn formula regardless of any additional conclusions.

As we will discuss later, Horn clauses make up Logic programs. When you are writing Logic programs, you are simply writing a set of Horn clauses.

Logic Programming Paradigm

Introduction

Just as functional programming paradigm is patterned from the formalisms of lambda calculus, logic programming is patterned from predicate calculus. Computer Scientists usually describe families of programming languages under the logic paradigm as a sub-paradigm of declarative programming (*declarative programming being any paradigm that is not imperative*).

Learning Outcomes

1. Create Prolog facts, rules, and queries
2. Explain the process of unification
3. Explain how proof search is used to respond to queries
4. Create recursive Prolog rules

For this section we will use the programming language Prolog as the representative of logic paradigm. Other logic programming families are answer set programming, ABYSS and Datalog.

Facts

There are three basic constructs in Prolog, **facts**, **rules** and **queries**. A **knowledge base** is a collection of facts and rules in the same way a c library or a python package is a collection of function definitions. Prolog programs are basically knowledge bases. Here's an example of a knowledge base:

```
firetype(charmander).  
firetype(charizard).  
watertype(squirtle).  
flyingtype(charizard).
```

Each line of code you can in this particular knowledge base is a fact. Just like facts, in logic, facts in Prolog are propositions that are known to be true. This means that your program knows four things. One of those are:

```
firetype(charmander).
```

In Prolog `firetype(X)` represents a predicate you've learned in discrete math, e.g. `firetype(x)`. And just like predicates, this means X is firetype. Therefore, the fact `firetype(charmander)` represents the proposition, "*charmander is firetype*"

So, to summarize, the fact `firetype(charmander)` is basically a representation of the proposition, `$firetype("charmander")$` where `firetype/1`⁸ is a predicate and `charmander` is a value assigned to the predicate. Every fact in your knowledge base represents propositions that can be assumed to be true. Facts that are not in your knowledge base represents propositions that you cannot assume to be true.

⁸`firetype/1` indicates a predicate named `firetype` with 1 argument.

You can also write non-predicate based propositions in your knowledge base. For example:

```
this_is_a_fact.  
axiom_1.
```

Facts, just like other basic Prolog constructs, are implementations of Horn clauses. Specifically, facts represent Horn clauses with one positive literal and nothing else. A fact like, `firetype(charmander)` can be written in the form of an implication as such:

$$\begin{aligned} \text{firetype}(\text{charmander}) &\equiv \text{firetype}(\text{charmander}) \vee \perp \\ &\equiv \top \rightarrow \text{firetype}(\text{charmander}) \end{aligned}$$

Horn clauses like these are called facts because, these clauses are always assumed to be true via modus ponens.

Rules

Aside from facts, you can also define rules in your knowledge base. To illustrate this, let's add one rule to our knowledge base.

```
firetype(charmander).  
firetype(charizard).  
watertype(squirtle).  
flyingtype(charizard).  
  
resistanttofire(squirtle) :- watertype(squirtle).
```

Rules are written with the syntax: **u :- v**. This is equivalent to the implication statement $v \rightarrow u$. Prolog rules are written using the “u if v”, the reverse of a conventional “if v then u” implication statement. A lot of people get confused here so just remember, `:-` is read as if. In prolog, we call the conclusion u as the rule **head** and the hypothesis v as the rule **body**.

A **rule** is an implementation of a definite Horn clause. It is a Horn clause with exactly one positive literal and one or more negative literal.

This will be more obvious if we convert the rule into its disjunction form.

$$\begin{aligned} \text{watertype}(\text{squirtle}) \rightarrow \text{resistanttofire}(\text{squirtle}) &\equiv \\ \neg \text{watertype}(\text{squirtle}) \vee \text{resistanttofire}(\text{squirtle}) \end{aligned}$$

Queries

We interact with a knowledge base by writing **queries** to Prolog. You can probably guess what this Prolog construct means just by looking at its name. Queries represent *questions* you ask Prolog. The answer to a question depends on what Prolog knows. And what Prolog knows is represented by the knowledge base. For example if you load the knowledge base we created earlier. We can ask Prolog the following question below. Writing the following will basically ask Prolog, “hey, is it true that charmander is firetype?”

```
?- firetype(charmander).
```

Based on the knowledge base loaded earlier Prolog knows that this proposition is indeed true. Therefore, it responds with:

true.

If Prolog is asked with the query

?- firetype(squirtle).

Prolog checks its knowledge base again. Realizing that none of the facts match this proposition, Prolog responds with:

false.

You can also ask Prolog a conjunction as a query. This is written multiple queries, separated by a comma.

?- firetype(charizard), watertype(squirtle).

A conjunction of queries can only be true if all queries in said conjunction are true.

true.

Prolog queries are also implementations of Horn clauses. When you provide a query to prolog, prolog tries to prove that the query is true using **proof by contradiction**. To do this, the query is negated, converting them to a goal. For example, the query `firetype(charizard), watertype(squirtle)`, is negated into the disjunction:

$$\neg \text{firetype}(\text{charizard}) \vee \neg \text{watertype}(\text{squirtle})$$

This negated query or goal is then combined into the knowledge base. If the knowledge base is unsatisfiable with the introduction of the goal, then that means that the goal introduced a contradiction. Therefore, the goal's negation (the original query), must be **consistent** with the knowledge base's assumptions. This ultimately means that it is **true** with respect to the knowledge base.

Here's an example, given the knowledge base:

p
q
r :- q

And the query:

?- r

To demonstrate the answer to the query easily, let's convert the clauses in the knowledge base into disjunctions and add the query as a goal⁹.

⁹When converting prolog rules into definite Horn clauses, the rule body is negated. This is because a rule body can be a conjunction, while a rule head can't. To ensure that the resulting clause is a valid Horn clause, we negate the body.

$$\begin{aligned}
 &(p) \wedge \\
 &(q) \wedge \\
 &(\neg q \vee r) \wedge \\
 &(\neg r)
 \end{aligned}$$

We then check if resulting Horn formula is satisfiable:

$$\begin{aligned}
 &(\top) \wedge \\
 &(\top) \wedge \\
 &(\perp \vee r) \wedge \\
 &(\neg r) \\
 & \\
 &(r) \wedge \\
 &(\neg r) \\
 & \\
 &(\top) \wedge \\
 &(\perp)
 \end{aligned}$$

This shows that combining our goal with the Horn formula leads to an unsatisfiable formula. Thus proving via contradiction that r is indeed true. With this Prolog appropriately responds `true`.

`true`.

Variables

Another important thing about Prolog constructs is that you can write them with **variables**¹⁰. When you write with Prolog facts or rules, you are implicitly creating a *universally instantiated predicate*. For example, the fact `pokemon(X)`¹¹, corresponds to the proposition, $\forall x(\text{Pokemon}(x))$. By adding this to the knowledge base, you are assuming that for any value x , `pokemon(X)` is true. Therefore, asking the query, `pokemon(charizard)` will yield a `true` response. Of course, any value supplied to the predicate `pokemon/1` will yield `true` because of the universal quantification.

```
pokemon(X).
```

```
?- pokemon(charizard).
```

```
true.
```

You can also use variables on queries. For example, if we use one of the previous knowledge bases and ask the query: `firetype(X)`.

```
firetype(charmander).
firetype(charizard).
watertype(squirtle).
flyingtype(charizard).
```

¹⁰Variables, in the context of logic programming, do not refer to the same variables in imperative programming. These variables represent, mathematical variables that can be free or bound.

¹¹Prolog variables must start with either a capital letter or an underscore.


```
resistanttofire(squirtle) :- watertype(squirtle).
```

```
?- firetype(X)
```

This query basically asks, which values when substituted to *X* in the predicate *firetype(X)* will yield true statements? This can be interpreted in natural language as “which Pokémon are fire type?” Therefore, this query will yield the response:

```
X = charmander
```

```
X = charizard
```

Variables used in rules allows the creation of richer knowledge bases. Instead of the rule *resistanttofire(squirtle) :- watertype(squirtle).* we can write a more general rule using variables:

```
firetype(charmander).
```

```
firetype(charizard).
```

```
watertype(squirtle).
```

```
flyingtype(charizard).
```

```
isresistantto(X,Y) :- watertype(X),firetype(Y).
```

```
isresistantto(X,Y) :- watertype(X),watertype(Y).
```

This introduces a more complicated rule *isresistantto(X,Y) :- watertype(X),firetype(Y).*

This rule’s premise is a conjunction of predicates *watertype(X)* and *firetype(Y)*.

If we imagine that the predicate, *isresistantto(x,y)* means “x is resistant to y”, the whole rule can be interpreted as

for all pairs of *X* and *Y*, *X* is resistant to *Y*, if *X* is water type and *Y* is fire type,

This statement, can be written as the following quantification statement:

$$\forall x \forall y ((watertype(x) \wedge firetype(y)) \rightarrow isresistantto(x,y))$$

By writing this rule, Prolog can infer the following facts:

```
?- isresistantto(squirtle,charmander).
```

```
true.
```

```
?- isresistantto(squirtle,charizard).
```

```
true.
```

```
?- isresistantto(squirtle,squirtle).
```

```
true.
```

If you ask Prolog a harder question like the following:

```
?- isresistantto(squirtle,X).
```

Prolog interprets this as “which values of *X* make the proposition: squirtle is resistant to *X*, true? Therefore, Prolog will look for the pokémon, squirtle is resistant to, therefore you with the output:

```
X = charmander
X = charizard
X = squirtle
```

You can even ask Prolog for all possible pairs of resistance relationships in the knowledge base. You can do this by using two different variables for each argument in the predicate.

```
?- isresistantto(X,Y).
```

Unification and SLD Resolution

Unification

There are three types of Prolog terms (Blackburn et al. 2006). By composing these terms you can express rich knowledge bases.

1. Constants. These can either be atoms (known to us as strings such as `squirtle`) or numbers (such as `24`).
2. Variables. Those that start with an underscore or any uppercase letter such as `X`, `Z3_4310`, and `List`.
3. Complex terms. These have the form: `functor(term_1, ..., term_n)`. We’ve seen examples of these in predicates and queries such as `firetype(charmander)` and `isresistantto(X,Y)`

The way Prolog is able to respond to interesting queries involving constants, variables and complex terms is through the **unification**. Unification algorithmically identifies **logically consistent substitutions** involving constants, variables, and complex terms. Unification in the context of Prolog works along the following rule:

Two terms unify if they are the same term or if they contain variables that can be uniformly instantiated with terms in such a way that the resulting terms are equal.

This definition gives us the unification of trivial cases such as the unification of constants `squirtle` and `squirtle`. Prolog also unifies the complex terms `watertype(squirtle)` and `watertype(squirtle)` and the variables `X` and `X`. On the other hand, the complex terms `watertype(squirtle)` and `watertype(blastoise)` will not unite.

Prolog also unifies the variable `X` with the constant `squirtle`. Although they are not the same, the variable `X` can be assigned to `squirtle`. By the same intuition, `watertype(X)` and `watertype(squirtle)` will also unify. Unification involving variables, create what is known as an instantiation. Unifying `watertype(X)` and `watertype(squirtle)` will create the instantiation `X=squirtle`.

On the other hand the complex terms `isresistantto(X,charmander)` and `isresistantto(squirtle,X)` do not unify since you cannot find a consistent instantiation of `X`. The instantiation `X=squirtle` evaluates to the terms `isresistantto(squirtle,charmander)`

and `isresistantto(squirtle,squirtle)`. The instantiation `X=charmander` makes the terms `isresistantto(charmander,charmander)` and `isresistantto(squirtle,charmander)`.

Let $x = \text{squirtle}$
`isresistantto(squirtle, charmander)`
`isresistantto(squirtle, squirtle)`

Let $x = \text{charmander}$
`isresistantto(charmander, charmander)`
`isresistantto(squirtle, charmander)`

Both scenarios cannot work because the constants `squirtle` and `charmander` do not unify with each other.

The process of unification can be summarized by the following^[1]:

Two terms a and b unify if and only if

1. a and b are constants, and they are the same number or atom
2. a is a variable and b is any type of term (in this case a is instantiated to b) or b is a variable and a is any type of term (in this case b is instantiated to a). This rule automatically unifies any pair of variables
3. a and b are complex terms and:
 1. They have the same functors and the same number of arguments
 2. all their corresponding arguments unify
 3. the variable instantiations are uniform or compatible (you cannot instantiate x to some constant a when unifying a pair and instantiate x to another constant b when unifying another pair of arguments)

You can demonstrate unification in the Prolog terminal using the predicate `=/2` (this means the `=` functor with two arguments).

```
?- =(squirtle,squirtle)
true.
```

```
?- =(squirtle,charmander)
no
```

```
?- =(squirtle,X)
X=squirtle
true.
```

```
?- =(X,Y)
X=_5071
Y=_5071
true.
```

```
?- =(watertype(X),watertype(squirtle))
X=squirtle
true.

?- =(f(g(X),X),f(Y,a))
X=a
Y=g(X)
true.
```

Unification with variables is an implementation of **universal instantiation**. Remember that a $p(X)$ in Prolog implicitly means $\forall x p(x)$. When you assume $p(X)$ in the knowledge base, you assume that $\forall x p(x)$ which implies via *universal instantiation* that $p(a)$ is also true (for some constant a).

When you have two variables unified to each other, they automatically unify because the actual variable name used does not matter at all. For example, $p(X)$ will unify with $p(Y)$, because there is no semantic difference between that universal quantifications $\forall x p(x)$ and $\forall y p(y)$.

Using the same intuition you'll find why the terms $p(X, a)$ and $p(b, X)$ will not unify. These terms represent $\forall X p(X, a)$ and $\forall X p(b, X)$, which can only consistently instantiate if $X = a$, $X = b$, and $a = b$.

Programming with unification

Unification is crucial with how one can write interesting logic programs. By creating knowledge bases that take advantage of unification, you can generalize structures based on the facts and rules of its characteristics. For example, the following is a knowledge base describing the characteristics of vertical and horizontal lines¹³:

```
vertical(line(point(X,Y),point(X,Z))).
horizontal(line(point(X,Y),point(Z,Y))).
```

Therefore, asking the query:

```
?- vertical(line(point(1,2),point(1,3)))
```

Will yield the response:

```
true.
```

It is indeed a vertical line. And the knowledge base makes sense, since any line that has the same x coordinate is vertical and any line that has the same y coordinate is horizontal.

We can even ask more general queries to Prolog such as:

```
?- horizontal(line(point(2,3),point(U,4)))
```

¹²The instantiations $X=_5071$ and $Y=_5071$ show that both X and Y are instantiated to the same dummy variable created by Prolog. This is also known as X and Y being aliased to each other, meaning that they share each other's instantiations.

¹³Note that the functors `line`, and `point` are not predicates. These are simply used as a way to structure the arguments. Only the outermost functors `vertical/1` and `horizontal/1` are considered predicates.

This query corresponds to asking Prolog for horizontal lines starting at (2, 3) and ends at a point with 4 as the y -coordinate. Prolog attempts the following variable unifications:

```
=(X,2)
=(Y,3)
=(U,Z)
=(Y,4)
```

Individually, these queries unify. But the unification creates inconsistent instantiations, namely, $Y=3$ and $Y=4$. These instantiations are inconsistent because 3 does not unify with 4. Since Prolog can't unify this query with any value for U (horizontal lines must have the same y -coordinate), Prolog responds:

```
false.
```

Selective Linear Definite Resolution

Prolog uses the algorithm called **Selective Linear Definite Resolution** (SLD Resolution) to answer queries efficiently. The process is equivalent to checking if the combination of the knowledge base clauses and the goal leads to an unsatisfiable formula. But this method provides a step-by-step process that can be easily implemented by computers.

Let's start with a simple example:

```
q(a).
s(b).
p(X).
?- p(a), p(b)
```

Given the query, $p(a), p(b)$, Prolog produces a goal by negating the query. The goal is now $\neg p(a) \vee \neg p(b)$. Since the goal is a disjunction, it must prove that a contradiction arises from both $\neg p(a)$ and $\neg p(b)$. This creates two separate sub goals $\neg p(a)$ and $\neg p(b)$. One of the aspects of SLD resolution is how it only *selects* one **subgoal** out of the conjunction of goals. Prolog selects the leftmost subgoal first, in this case, $\neg p(a)$.

Prolog searches the entire knowledge base from top to bottom, and tries to unify the current subgoal with any fact or rule head in the knowledge base. A successful unification, resolves the subgoal away. In this case, $\neg p(a)$, unifies with $p(X)$, creating the instantiation $X = a$.

If the resolution is unclear, you can write $\neg p(a)$ and $p(X)$ as separate clauses¹⁴.

$$\begin{aligned} p(X) \vee \perp \\ \neg p(a) \vee \perp \\ \perp \vee \perp \end{aligned}$$

¹⁴ $p(X)$ and $\neg p(a)$ are complements of each other through unification. Remember that $p(X)$ in Prolog implicitly means $\forall x p(x)$. Through universal instantiation, we know that $\forall x p(x)$ implies that $p(a)$ is true. And we know that $p(a)$ is a complement of $\neg p(a)$.

This resolves $\neg p(a)$, cancelling it out. Thus, leaving $\neg p(b)$ as the only remaining subgoal to be resolved. Prolog repeats the process for $\neg p(b)$, unifying with $p(X)$ with the instantiation $X = b$. Resolving $\neg p(b)$ leaves all subgoals resolved. With all subgoals resolved and refuted, the query is proven true by contradiction.

Let's try an example that involves rules on the knowledge base (example from Blackburn et al. (2006)).

$f(a).$
 $f(b).$

$g(a).$
 $g(b).$

$h(b).$

$k(X) :- f(X), g(X), h(X).$

With the query:

$?- k(Y).$

The goal produced from the query is $\neg k(Y)$.

$$\frac{\text{Current Goal}}{\neg k(Y)}$$

Prolog goes through the entire knowledge base from top to bottom finding unification with the head of $k(X) :- f(X), g(X), h(X)$. The resolution of a rule and a goal leaves a resolvent which serves as new subgoals¹⁵.

$$\begin{aligned} & \neg k(Y) \vee \perp \\ & k(X) \vee \neg f(X) \vee \neg g(X) \vee \neg h(X) \\ & \neg f(X) \vee \neg g(X) \vee \neg h(X) \end{aligned}$$

With $\neg k(Y)$ resolved away, and the resolvent added to the goals, we are left with the following goal.

$$\frac{\text{Current Goal}}{\neg f(X) \vee \neg g(X) \vee \neg h(X)}$$

With this new goal, Prolog starts resolving the leftmost subgoal, $\neg f(X)$. This subgoal unifies with two facts, $f(a)$ and $f(b)$. Whenever this happens, Prolog creates two branches (one for each possible unification). Prolog tries to complete the refutation one branch at a time in a depth first manner. The other branch will be revisited once one branch is completed.

¹⁵This resolution is just modus tollens.

For the unification to be consistent, the other subgoals must also unify with respect to the instantiation $X = a$. With this, $f(X)$ is resolved away, and the goal narrows down to the following:

$$\frac{\text{Branch } X = a \text{ goal (current)} \quad \text{Branch } X = b \text{ goal}}{\neg g(a) \vee \neg h(a) \quad \neg g(b) \vee \neg h(b)}$$

In this case, it tries to complete the branch formed from $X = a$ first. The subgoal $\neg g(a)$ unifies with $g(a)$ in the knowledge base.

$$\frac{\text{Branch } X = a \text{ goal (current)} \quad \text{Branch } X = b \text{ goal}}{\neg h(a) \quad \neg g(b) \vee \neg h(b)}$$

The subgoal $\neg h(a)$ does not unify with any clause in the knowledge base. Therefore, it cannot be resolved away. This means that the branch for $X = a$ completes without refutation.

$$\frac{\text{Branch } X = a \text{ goal (not refuted)} \quad \text{Branch } X = b \text{ goal}}{\neg h(a) \quad \neg g(b) \vee \neg h(b)}$$

From here, Prolog proceeds to resolve the goal of the $X = b$ branch. In this branch both subgoals do unify, with $g(b)$ and $h(b)$. This resolves the goal which completes the branch.

$$\frac{\text{Branch } X = a \text{ goal (not refuted)} \quad \text{Branch } X = b \text{ goal (refuted)}}{\neg h(a) \quad \perp}$$

Since one branch is refuted, Prolog responds to the query with `true`. Prolog also shows which instantiations/branches lead to a refutation. In this case, $X = b$ is refuted and since $X = Y$:

`Y = b,`
`true.`

Advanced Constructs in Logic Programming

Recursive Definitions

Similar to functional programming, logic programming represents repetition using recursion. While functional programming makes heavy use of recursive functions to implement complex behavior, logic programming languages like Prolog uses recursive rules to model complex structures. For example: consider the following knowledge base:

```
is_ancestor(Parent, Child) :- is_parent(Parent, Child).
is_ancestor(Ancessor, Descendant) :-
    is_parent(Parent, Descendant),
    is_ancestor(Ancessor, Parent).

is_parent(juan, francisco).
is_parent(cirila, francisco).
```

```
is_parent(teodora, jose).
is_parent(francisco, jose).
is_parent(brigida, teodora).
is_parent(lorenzo, teodora).
```

You'll notice that the rule `is_ancestor` is special since one of its goals is itself. The query below will yield a true response due to the recursive nature of the `is_ancestor` rule.

```
?- is_ancestor(juan, jose).
```

The query will match both rule heads, but the first instance (the base case) will lead to an unresolved goal since `is_parent(juan, jose)` cannot be resolved. On the other hand the second instance of the rule (the recursive case), will lead to a propagation of goals that will resolve.

Numbers in Logic Programming

Since logic calculus is a formalism for the foundation of mathematics, how do numbers emerge from predicates and propositions?

This is also another concept shared between, logic calculus and lambda calculus. You can represent numerals (specifically integers) using Peano's axioms:

0 is an integer.

The successor of an integer, denoted by $s(n)$ is also an integer.

You can represent these axioms as a knowledge base:

```
int(0).
int(s(X)) :- int(X).
```

This knowledge base will then define all the possible natural numbers out there, demonstrated by the query:

```
int(X)
X = 0
X = s(0)
X = s(s(0))
X = s(s(s(0)))
...
```

Using this representation, you can then define arithmetic operations such as addition and multiplication (also based on Peano's axioms) (Hosch 2010).

$$a + 0 = a$$

$$a + b = c \rightarrow a + (b + 1) = (c + 1)$$

$$a * 0 = 0$$

$$a * b = d \wedge a + d = c \rightarrow a * (b + 1) = c$$


```
int(0).
int(s(X)) :- int(X).
```

```
add(A,0,A).
add(A,s(B),s(C)) :- add(A,B,C).
```

```
mult(_,0,0).
mult(A,s(B),C) :- mult(A,B,D), add(A,D,C).
```

While this representation of numbers is a good way of demonstrating how numbers can be defined using logic, it's not really that usable. We want to numbers to be easily readable, that's why we use base-10 Arabic numerals to represent numbers. Prolog does have a builtin representation for numbers. Numbers are accepted terms in the form of constants. You can also apply some predicates to numbers like `=`, `<`, `>`¹⁶.

```
positive(X) :- X > 0.
?- positive(2).
true.
```

Unfortunately, these representations are also very limited. Predicates like `=`, `<`, and `>` break the logical nature of Prolog. If you try to use these predicates with uninstantiated variables, you will end up with an error.

```
?- X = 2.
Arguments are not sufficiently instantiated
In:
[1] 2>_2258
```

To solve this limitation, the library `clp(fd)` and `clp(z)` was created. The library `clp(fd)` stands for **Constraint Logic Programming Over Finite Domains**. This library offers predicates that can be used to apply logical reasoning on integers. This library was further refined to a more complete and more advanced library called `clp(z)` or **Constrained Programming Over Integers**.

These libraries include special predicates known as constraints. **Constraints** are predicates that restrict a variable to a specific set of values. Constraints are usually applied to *numerical variables* to define the domain of said variable. The libraries, `clp(fd)` and `clp(z)` include the equality and inequality constraints (Triska 2026).

- $X \#= Y: x = y$
- $X \#< Y: x < y$
- $X \#> Y: x > y$
- $X \#\neq Y: x \neq y$
- $X \#>= Y: x \geq y$
- $X \#<= Y: x \leq y$

¹⁶Note that equality and inequality operators are also predicates. In the examples here they are written as infix operators, but you can still use them using the prefix functor syntax (i.e. `=(2,3)`)

To import libraries in your knowledge base, use the headless rule syntax¹⁷.

```
:- use_module(library(clpfd)).
```

You can import it on the REPL by directly writing the `use_module` predicate as a query. With the library loaded in the repl, you can use the constraints on your queries.

```
?- use_module(library(clpfd)).  
true.
```

When you use the `#<` constraint with a variable, it restricts said variable's domain to satisfy the constraint.

```
?- X #< 3.  
X in inf..2.
```

In the example above, with the constraint `X #< 3`, the integer variable `X` can only have values in the range $[-\infty, 2]$.

When used with numerical expressions, you can use Prolog as solver.

```
?- 4 #= 2*X + 2.  
X = 1.
```

Using constraint programming, will be limited when used as a solver since the domain is finite and you the answers are based on relations. If the solution to your equation is not an integer, then the variable `X` doesn't have a valid integer that satisfies the constraint¹⁸. In such case, Prolog will respond with `false`

```
?- 0 #= 2*X + 3.  
false.
```

You can combine multiple constraints into a conjunction to further control the domain of an integer. In the example below, the values of `X` can only be in the range $[3, 11] \cup [13, \infty]$ ¹⁹.

```
?- X #>= 3, X #\= 12.  
X in 3..11\13..sup.
```

You can also directly use the `in` predicate to create constraints over a domain expression. In the example below, the domain ranges `0..10` and `9..12` are combined using the union operator, `\/`.

```
?- X in 0..10 \/ 9..12.  
X in 0..12.
```

To enumerate values in a domain, you can add the predicate `indomain/1` with a conjunction.

```
?- X in 0..10, 1 #\= X mod 2, indomain(X).  
X = 0 ;
```

¹⁷Older implementations, like `swi-prolog` comes with `clpfd` but not `clpz`. Newer implementations like `scryer-prolog` come with both.

¹⁸There are libraries that support constraint logical programming over real numbers like `clpr` which also comes bundled with `swi-prolog`.

¹⁹`inf` stands for infimum, the smallest value in the domain and `sup` stands for supremum, the largest value in the domain.

```

X = 2 ;
X = 4 ;
X = 6 ;
X = 8 ;
X = 10.

```

Data Structures

As discussed before, Prolog support compound terms in its syntax. Depending on where a term appears in Prolog code, it can be treated as a predicate with arguments, or a term representing data.

```

a(X).
b(c(d(Y)), e).
f(X) :- g(h), i(j).
k.

```

If a Prolog term appears as a fact, the head of a rule, or in the body of the rule, it is treated as a predicate with arguments. In the example above, `a/1`, `b/2`, `f/1`, `g/1`, `i/1`, `k/0` are considered as predicates. On the other hand, if a term appears as a predicate argument or a term argument, then it is treated as a term representing data. In the example above, `X`, `c(d(Y))`, `d(Y)`, `Y`, `e`, `h`, `j`, are considered terms representing data.

Compound data

Compound terms can be used to represent compound data, for example, you can represent a compound data structure that represents identity, using the generic term `id(Name, Age)`.

```

:- use_module(library(clpfd)).

person(id(artman, 16)).
person(id(bartman, 19)).
person(id(cartman, 21)).
adult(Name) :-
    person(id(Name, Age)),
    Age #>= 18.

```

Lists

Prolog uses a special representation for lists. An **empty list** is represented by the special atom `[]`. A special functor can be used to denote recursively from here. For example, we can define use `list/2` to denote any list. Any term defined as `list(Head, Tail)` is a *list*, if `Tail` is a *list*.

With this you can represent a list with *one element* as `list(elem1, [])`. You can define a list with *two elements* as `list(elem1, list(elem2, []))`.

As the number of elements grow, this representation can be cumbersome, so Prolog uses a special syntax to lists with any number of elements using comma separated terms. For example, a *list with four elements* can be written as `[elem1, elem2, elem3, elem4]`. Prolog still allows us to use the *head-tail* pattern for defining lists as `[Head|Tail]`. Such a list is valid if `Tail` is also a *valid list*. For example, the four element list can be represented as `[elem1 | [elem2, elem3, elem4]]`.

```
?- ([elem1, elem2, elem3, elem4], [elem1 | [elem2, elem3, elem4]]).
true.
```

You can also combine, comma separated lists with the head-tail pattern as such:

```
?- ([elem1, elem2, elem3, elem4], [elem1, elem2 | [elem3, elem4]]).
true.
```

You can use the [Head|Tail] pattern to create interesting predicates involving lists. For example, you can find the length of a list by creating the following rule:

```
:- use_module(library(clpfd)).

list_length([], 0).
list_length([Head|Tail], Length) :-
    Length #= Tail_length + 1,
    list_length(Tail, Tail_length).

?- list_length([a,b,c,d],X).
X = 4.
```

This predicate is very similar to the way a list's length is calculated in functional programming. But it can be used in more interesting ways. For example, we can create the predicate `list_nth_element/3`. Given, List, N, and Elem, `list_nth_element(List, N, Elem)` is true if Elem is found on index N in the list. To create this predicate, we first define a helper predicate called, `list_take_n/3`. Given List, N, Leftover, `list_take_n(List, N, Leftover)` is true if Leftover is the list you end up with after taking N elements from List.

```
:- use_module(library(clpfd)).

list_take_n(List,0,List).
list_take_n([Head|Tail], N, Leftover) :-
    N #> 0,
    M #= N - 1,
    list_take_n(Tail, M, Leftover).

list_nth_element(List, N, Elem) :-
    list_take_n(List, N, [Elem|Tail]).
```

With this `list_nth_element/3` created, we can find the nth element of a list.

```
?- list_nth_element([a,b,c,d,e],2,X).
X = c.
```

But at the same time, we can also use `list_nth_element/3` to check which indices an element is at.

```
?- list_nth_element([a,q,c,q,e],Index,q).
Index = 1 ;
Index = 3 ;
false.
```

We can write such queries because we are working with predicates that act as *pure relations*. Pure relations, unlike functions *do not have a set input and output*. A correctly written predicate should work no matter which argument is replaced by a variable.

Advantages and Disadvantages of Logic Programming

Advantages

Logic programming shares a lot of similarities with functional programming. It also shares its advantages and disadvantages as well. Both paradigms offer a safer and more consistent framework since they are both patterned from mathematical formalisms. Functional programming has lambda calculus and logic programming is based on *first-order predicate calculus*.

Being non-imperative also gives them an edge of automatically being *immune to the perils of state* and at the same time being *prone to the perils of its absence*.

Beyond the general advantages of declarative programming, Logic programming can be a powerful solver in problems like **automated theorem proving**, **constraint related search problems**, **domain specific expert systems** and more. The straight forward way of listing facts and rules makes it suitable for representing complex information that can be usually found in these use cases. The beauty of unification and SLD resolution shines on these domains as they often require, complex representation involving multiple interconnected rules, recursive structures, and constraints.

Logic programs also use predicates as *pure relations*. Because of this, the predicate definitions can support wide uses.

Disadvantages

Logic programming's way of expressing knowledge gives it a lot of niche uses. Most of the distributions Prolog are admittedly meant for a *limited* use cases only. You are not going to use it for web-development, embedded systems, or scripting.

Logic programming's disadvantages are also similar to functional programming, but can be much *worse*. The obvious *inefficiency* due to the absence of state is much more evident in logic programming because of the thorough approach of backtracking in SLD resolution. The *strangeness* of the paradigm itself as compared to the imperative way of thinking is also felt more in imperative programming. At least functional paradigm and imperative paradigm share the concept of functions Prolog only has predicates. To get used to writing logic programs, you have to get used thinking in terms of *relations* and *declarative definitions*.

Just like functional programming though, the spirit of logic programming can be found in other paradigms through the existence of unification libraries. Although logic paradigm is admittedly less relevant than other paradigms, its strange features are definitely useful and worth studying.

²⁰The extra false branch is a result of SLD resolution unifying the query to both the fact and the rule head in the knowledge base.

Object-Oriented Programming Paradigm

Introduction

As procedural programming became more and more mainstream, computer scientists started to notice the issues behind states and side effects. Some languages offered a complete paradigm shift, completely abandoning the notion of state. This exodus formed the alternative paradigm family, declarative paradigm. Other languages however, went on a different direction. These languages offered a solution to fixing state by introducing richer features and an intuitive design. From this approach the paradigm object-oriented programming was born.

Learning Outcomes

After this discussion you should be able to

1. Explain how programmers started to focus on writing maintainable code
2. Differentiate between the object and the class
3. Explain OOP's philosophy, the surface vs. the volume
4. Explain what abstraction means
5. Describe the fundamental concepts: encapsulation, inheritance, and polymorphism

As time went by and as technology evolved, computer scientists noticed new issues on the code they were writing. The goal of building the most optimized machine code became less and less of a priority. Programmers had, under their disposal, faster and better hardware. As the demand for complex systems grew, the priority shifted from *code that runs*, to *code that was intuitive*. As programmers started to build bigger and bigger systems, they started to notice the issues in terms of maintainability.

This shift in focus can be seen from the extremes of procedural paradigm such as the programming language Assembly. Assembly code was not really built to be intuitive. It was built to reliably work. Assembly contained mechanisms to move data around and to control program flow. These features were enough for that time since the objective of programming was to write efficient code that bulky slow CPUs understand. Assembly programmers did not have time to worry about code elegance or intuitiveness.

The landscape started to change when hardware started becoming better and software started becoming more complex. Speed and memory wasn't that much of an issue anymore, so computer scientists' focus shifted towards the issue of maintainability. Software became bigger and more complex and understanding other programmers' code became more and more difficult. In fact, understanding your own code was also becoming difficult. Because of this writing code that a computer understands isn't enough anymore, a good programmer must write code that humans understand as well. Code shifted from computer centered to human centered.

But instead of redesigning the concept of imperative paradigm to solve maintainability, object-oriented programming sought to build on top of the features of imperative programming. State despite its known issues, still exists but OOP gave imperative programmers extra tools to protect

code from being carelessly mutated. Because of this OOP stayed under the imperative family since the programmer is still focused on telling the computer what to do using assignment statements and mutation.

Fundamental Concepts of OOP

Object-oriented programming is usually defined using its four core design principles:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

It is said that any programming language that contains these mechanisms is an object-oriented programming language. But calling the entirety of a programming language, OOP can be problematic since you are free to write code written in an “OOP language” without actually using these design principles. At the end of the day OOP is not merely a language classification, but a paradigm. Some programming languages are indeed intentionally written to be used for OOP design, but these classifications are less important than judging if the code itself written by the programmer adheres to OOP’s design principles.

The Object

Let’s start with the star and the building blocks of object-oriented programming. What exactly is an object?

An object is a living organism in your code. To grasp the capabilities and limitations of an object we will anthropomorphize objects. Treating an object as an organism will guide you on how you use objects effectively.

Just like any creature an object has both form (attributes/fields) and behavior (methods) (this is one of the reasons why C’s `struct` is not an object since `struct` instances only have form). Objects are written to be representations of real world nouns such as a person or an employee or a file. The best way to design objects is to simulate the real world form and behavior of what these objects represent. Think of objects as the **representatives** of things in your code. Since you cannot shove an actual employee in your system, you instead create a representative of that employee using an employee object. If you start thinking about objects like representatives instead of mere data holders you’ll be able to design a safe and elegant object.

The Class

Often discussed alongside an object is the class. There’s usually a lot of confusion when identifying the difference between a class and an object, and it is probably because in the universe of your code, the class and the object will have the same name.

A class is the specifications for the creation of objects. If you want to give a class a more proactive role, you can think of the class as the factory that builds objects. There are a lot of analogies out there to illustrate the relationship of classes and objects. For example, a star shaped cookie cutter (class) that makes star shaped cookies (objects). If you want to make star shaped cookies you use

the star shaped cookie cutters. You can also call objects as instances of a class. In fact that's what I call objects most of the time. For example, the Cash class, \$2.75 is an instance of a Cash object. Or a cat named Garfield is an instance of a class called Mammal.

If an object is a representation of a tangible real world object, then the class is the conceptual type/category of that real world object.

The following class diagrams are the representations of a book and an Employee. The attributes of the book are title, author, publishDate, and pages. These attributes also simulate the form of a real world book. This object also has methods called ISBN() and numPages(). The attributes of an employee are name, string, and salary, and it has a method called reassignJob().

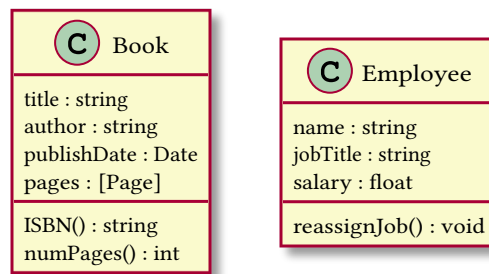


Figure 2: Class diagram of a book class

The following are representations of objects:

```

book1{
title: "Corpus Hermeticum"
author: "Hermes Trismegistus"
publishDate: Dec 12, 2008
pages: .....
}

book2{
title: "Behold a Pale Horse"
author: "Milton William Cooper"
publishDate: Dec 1, 1991
pages: .....
}

employee1{
name: "Rubelito Abella"
jobTitle: "Instructor 1"
salary: [REDACTED]
}
  
```


The Surface and the Volume

Data Hiding and the Interface

Before we play around with some code, let's dive deep into the philosophy of object-oriented paradigm. The base premise of OOP is the concept called **data hiding**. And this formal OOP term describes what I mean when I say that, what OOP did for imperative programming was to allow the programmer to create artificial boundaries between irrelevant data and behavior. In the eyes of an OOP design, procedural code is a mix of data and functions arbitrarily tossed in a spaghetti of mutations and side effects.

OOP's mechanism to create boundaries in the form of objects brought structure to imperative programming. All the attributes of a Book object doesn't need to be accessed by the methods of a LibraryCard object. That's because in the real world, a library card doesn't need to know what's written on page 69 of some book. Some of the data in Book should be **hidden** from a client object LibraryCard.

Object-oriented programming's obsession to simulate real world objects is not caused by its dream to become an exact representation of the real world. Object-oriented programming obsesses over structure and simulation because it is a necessity for human comprehension and therefore, maintainability. Systems need good structure not because well-structured code is pleasant and elegant to look at, but because our feeble minds can't process poor structure efficiently. In fact the reason why we call a well-structured piece code, "elegant" is because our tiny limited minds can process it well.

Trying to read this code would be comparable to looking at some Lovecraftian cosmic horror. If such a code exists it'll probably contain the secrets of the universe.

The process of modeling elegant object representations is basically determining what the **public interface** of that object may be. The interface of an object is the set of attributes and methods (methods are what we call functions that inside a class) that other objects can use to interact with it. The attributes and methods that are not in the interface is essentially hidden information, inaccessible from the client objects. For example, given a Book object that interacts with a LibraryCard object, the LibraryCard object's interface to interact with the book may only contain, title and author, because this is all the information it needs to properly do its job. You can even hide those attributes and let the LibraryCard only interact with the Book using a borrow() method which lists the Book as borrowed in the library card. It doesn't need access to the attributes of the book to do this. It interacts with the Book itself as a whole.

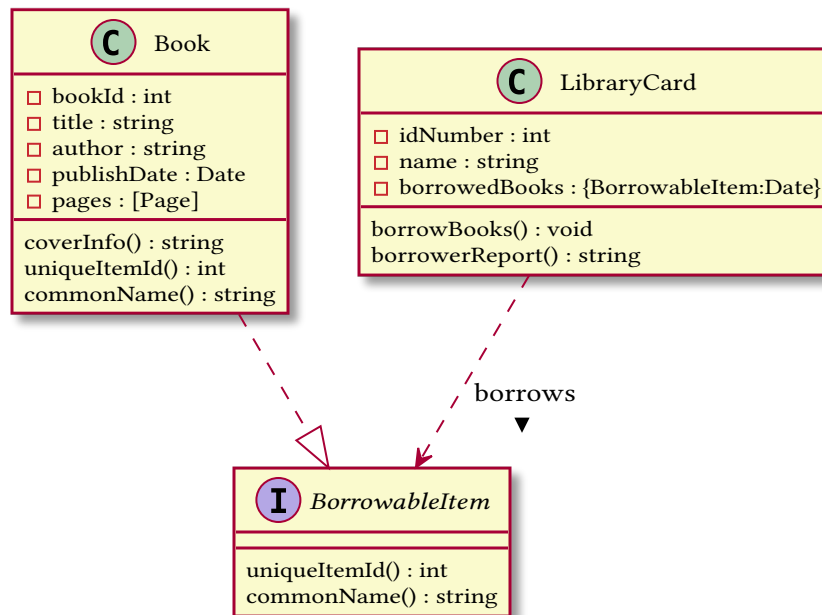


Figure 3: Class diagram of a book class

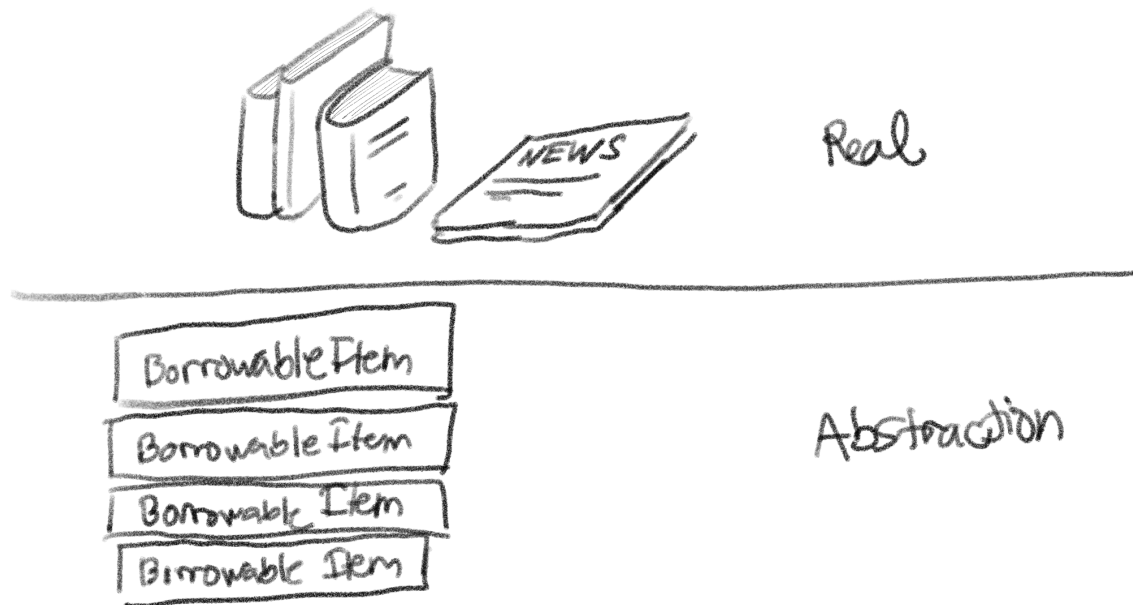


Figure 4: abstraction borrowable item

Abstraction of Objects

Creating interfaces like these provide OOP with the mechanism to create **abstractions**²¹ in the object level.

²¹The term abstraction is actually quite overloaded in OOP and CS in general. The “abstraction” I’m talking about in this context is the concept of abstract interface.

An abstraction in computer science is basically a model of computation that is free from its implementation. In the same way that functional programming creates abstractions of mathematical functions by writing lambdas without side effects, OOP creates abstractions of objects using interfaces that don't specify the exact implementation of an object.

In the example above, the interface `BorrowableItem` is an abstract representation of *something from the library that can be borrowed*. An interface like `BorrowableItem` contains method names and type signatures, but it doesn't actually contain code. That is because a `BorrowableItem` is an abstract representation. We are not supposed to care about the implementation of the methods `uniqueItemId()` and `commonName()` all we should care about is that `uniqueItemId()` should return some `int` and `commonName()` should return a `string`.

The reason why this structure still works, is because we have a concrete class called `Book` which is a **realization** or an **implementation** of `BorrowableItem`. A book is *something from the library that can be borrowed*. Because a `Book` is a `BorrowableItem`, it must also behave based on the specifications of a `BorrowableItem`. Meaning it must contain the methods `uniqueItemId()` and `commonName()` (which should also have the same type signature as the methods of `BorrowableItems`). Since `Book` is a concrete class its methods `uniqueItemId()` and `commonName()` should be implemented (meaning there should be code inside these methods).

Why even go through all this trouble? If *something from the library that can be borrowed* is an abstract idea, what is the point of modelling its representation? This looks like extra code just to represent something that doesn't really have an exact form in the real world.

For the current structure we created, this feels like extra code because our system is small enough right now. But imagine if our system grows, and we need to incorporate other things from the library that are not books but can be borrowed. For example, a library also contains periodicals that you can borrow as well, and these periodicals do not follow the form of the book. You need a different representation for a periodical, therefore you need to create a new concrete class called `Periodical`. Since a periodical is also *something from the library that can be borrowed*, a periodical is another **realization** of `BorrowableItem`. And with the tiny effort of writing the implementation of a periodical (including the realized methods `uniqueItemId()` and `commonName()`), we added an extra interaction that allows a `LibraryCard` to borrow periodicals as well.

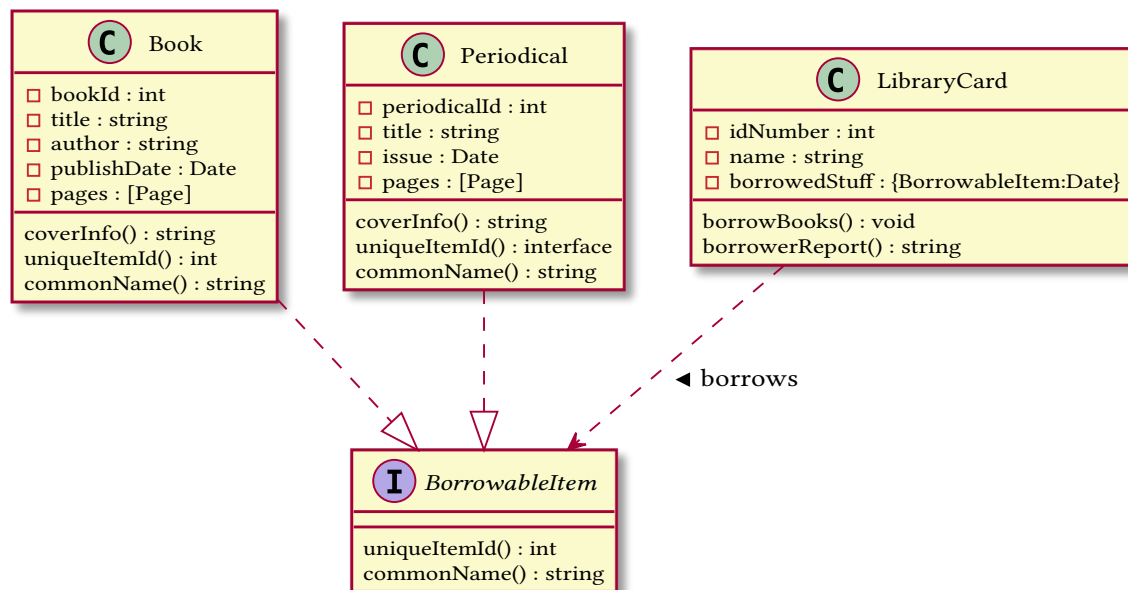


Figure 5: Class diagram of a book class

The Interface and the Implementation

Let's summarize what we learned so far by discussing how these capabilities characterize the philosophy of OOP. The paradigm aims to solve the issues of state and maintainability by allowing programmers to create boundaries between its mix of attributes and methods. The boundaries you enforce are basically the object structure you create. A library card name shouldn't mix with a book title, so we put a boundary between them by **encapsulating** them into their respective objects.

You can imagine these objects as amorphous blobs with surface separating the methods and attributes inside it from other things in your code. These amorphous blobs have volume and surface. The volume of these blobs represent the **implementation** of these objects and the surface of these blobs represent the **interface** of these objects. The interaction, between two objects is characterized by the surfaces of objects, the implementation. The volume of the object shouldn't dictate how the objects relate to each other, in fact everything inside the object should be inaccessible to other objects. Objects should only see each other's surface. This means that the interaction between objects should be defined by their interface not their implementation.

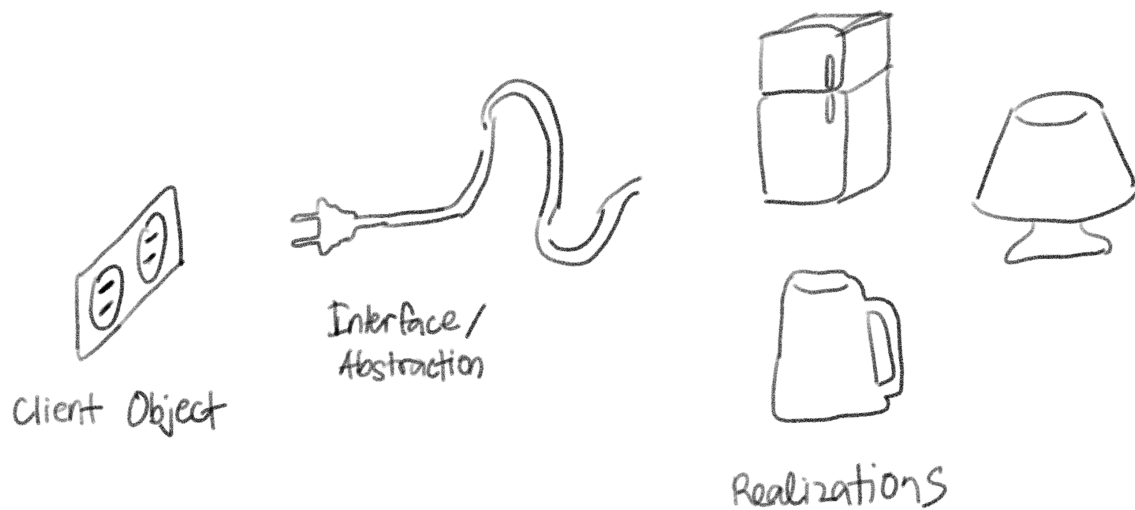


Figure 6: Abstraction borrowable item

Your job as an OOP programmer is to make sure that the complexity of the surface grows slower than the complexity of the core. This means that as your system evolves, changes that happen in the core, the implementation hidden inside each object, (as much as possible) shouldn't affect the surface, the interfaces of each object. This is what harmony and elegance in OOP means. Objects interact with each other seamlessly, and intuitively, regardless of what their inside look like.

Fundamental Concepts of OOP

Encapsulation

One of the most important design principle of object-oriented programming is the concept called encapsulation. I've said it again and again, and I don't mind saying it again right now, oop's innovation that made the paradigm a solution to the issues of state is its mechanism to construct boundaries wherever you want (you should want to put it between irrelevant data). This mechanism is also called **encapsulation**. Here are a few important points to remember:

- Encapsulation, when done correctly, makes your system approach a more accurate simulation of the real world. The more you encapsulate related data and methods, the more you'll create cohesive classes that have definite and indivisible purpose.
- You should **encapsulate what varies**, meaning, things that always change should be encapsulated deep into the structure of your code. This will help with maintainability since the changing isolated data or behavior will have less impact to the whole system.
- Encapsulation means both attributes and behaviors. Concrete objects should be given the responsibility of implementing their own behavior. This means that a method that describes the behavior of a certain class should belong to that class.

Which implementation is better?

Tax rate as a global variable

```

TAX_RATE = 1.1

product1_price = 100
product2_price = 35
exempt_product_price = 22 #exempted from tax

final_price += product1_price * TAX_RATE
final_price += product2_price * TAX_RATE
final_price += exempt_product_price

```

Tax rate hard coded for every instance of use

```

product1_price = 100
product2_price = 35
exempt_product_price = 22 #exempted from tax

final_price += product1_price * 1.1
final_price += product2_price * 1.1
final_price += exempt_product_price

```

A function called `taxedPrice()` which calculates tax

```

TAX_RATE = 1.1

def taxedPrice(price):
    return price * TAX_RATE

product1_price = 100
product2_price = 35
exempt_product_price = 22 #exempted from tax

final_price += taxedPrice(product1_price)
final_price += taxedPrice(product2_price)
final_price += exempt_product_price

```

Hardcoding tax calculation for every instance is the least encapsulated solution. This solution is the least future-proof version since changes in tax rate will require changing every instance of tax calculation manually as well. To improve upon this we can encapsulate tax rate into a global variable. Anytime there are changes to the tax rate, we simply change the value of `TAX_RATE` and all tax calculations will be updated as well.

This can be improved even more by encapsulating the tax calculation deeper into its own helper function. This version makes it more resilient to tax policy changes. Maybe in the future, tax calculation becomes more complex than simple multiplication. If this does happen our code is ready to accept the change by simply changing the body of `taxedPrice()`.

As seen here, we see how tax calculation is something that has potential to change. It is volatile. As mentioned earlier, it is prudent to encapsulate volatile code to make it easier to update.

Inheritance

Another important design principle in OOP is the concept of **inheritance**. Inheritance is the concept in which the definition of a class is derived from another class. An existing class, called the **super class** (also called the **base class** or the **parent class**) passes all visible attributes and methods to a **subclass** (also called the **derived class** or the **child class**).

The concept of inheritance is also a representation of the real world. You use inheritance to represent generalizations and specializations. A super class is a generalization of a subclass and a subclass is a specialization of a super class.

In this example the supertype animal is a generalization of the subtype mammal. Although it isn't shown, Mammal will also have the attributes name and weight and the method sound() since it inherits these from the parent class. Mammal has a method of its own called lactate() which it doesn't share with animal.

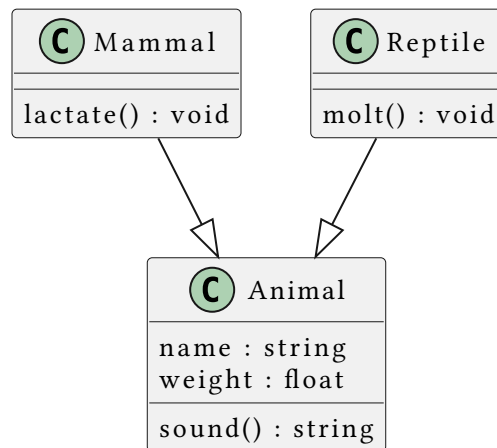


Figure 7: Inheritance

A subclass can also be a super class for another class. This is used to represent specializations of specializations.

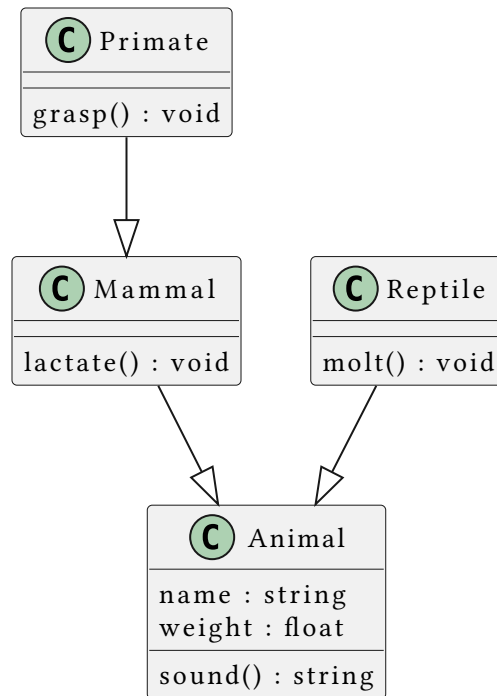


Figure 8: Inheritance

The class primates will then inherit all visible attributes and mehtods of Mammal which include those that are inherited from Animal.

Some programming languages will allow you to add restrictions to the inheritance of an attribute or a method. Languages like C++ or Java does this using the modifiers `private` and `protected`

super class visibility	public derivation	protected derivation	private derivation
public	public	protected	private
protected	protected	protected	private
private	<i>not inherited</i>	<i>not inherited</i>	<i>not inherited</i>

Polymorphism

Polymorphism literally means *multiple forms*. One of the core philosophy of OOP allows object instances to exist in multiple forms. What this means code-wise is that the types of object instances can be decided during runtime.

Compile-time polymorphism

There's also another type of polymorphism that is not necessarily shared by all OOP languages, compile-time polymorphism. This is basically the feature where multiple functions can have the same name as long as they have different parameter type signatures. We also call this feature as method overloading or dynamic dispatch.

Run-time polymorphism

Run-time polymorphism on the other hand, is basically achieved using specialization and realization relationships between objects. This is usually what Polymorphism refers to in the scope of OOP.

For example, An object instantiated to be of type `Primate` is also an instance of `Animal` because a primate is just a specialization of an animal. This is a reflection of how the real world works because a primate is indeed an animal. On realization relationships like a `Book` and a `BorrowableItem`, the same is also true, because a book is also something that can be borrowed. Realization and specialization relationships guarantee that you can interact with a subtype as its super type, and you can interact with concrete class as its abstraction.

SOLID Objects

Introduction

SOLID is an acronym describing design principles for creating OOP systems. By committing your code to these principles you will naturally build systems that don't only work perfectly, but work elegantly.

Learning Outcomes

1. Design methods and classes with exactly one responsibility
2. Design extension to classes to incorporate new behavior
3. Design proper realization and specialization relationships
4. Design purposeful abstractions and abstract methods
5. Design systems that interact through abstractions

Briefly, these five principles are:

- **Single Responsibility Principle** - Objects should have cohesive and complete responsibilities. It shouldn't be aware of knowledge it doesn't need, and it shouldn't perform responsibilities that are irrelevant from it.
- **Open/Closed Principle** - Classes should be open to extension and closed to modification. Instead of changing the form and behavior of an existing class, you should extend the class
- **Liskov-Substitution Principle** - Substituting objects by their subtypes/realizations should always work.
- **Interface Segregation Principle** - A client shouldn't be forced to implement methods that it doesn't use
- **Dependency Inversion Principle** - Object relationships should depend on abstractions instead of implementations

Single Responsibility Principle

The GOD Class

One of the canonical examples of violations against SRP is the concept known as the **god class**. A god class is a class that basically contains all the attributes and methods of the whole system. You'll

recognize these god classes as those classes that control the behavior of objects (they contain the implementation of client objects' behavior). These god classes are also aware of all the objects secrets (they expose and manipulate private attributes and methods).

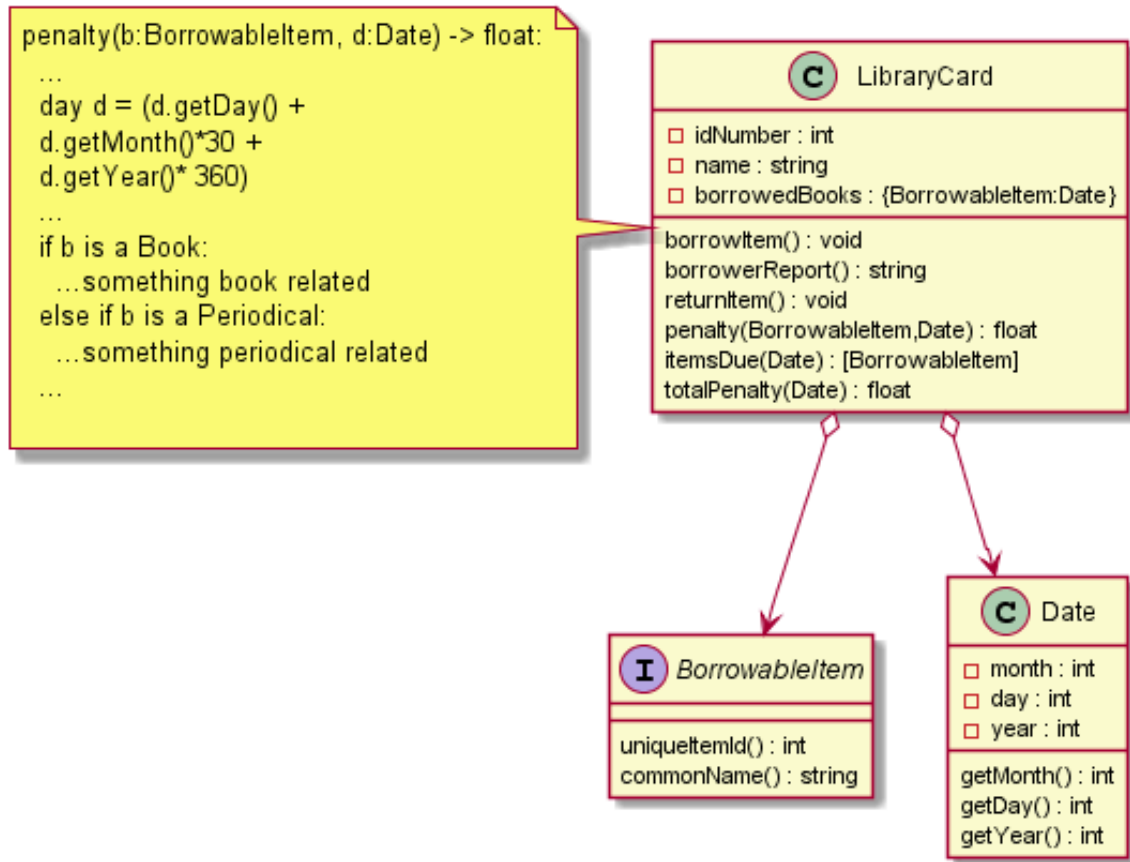


Figure 9: God class library card

On the example above LibraryCard is a god class since it exposes the secrets of Date by forcing the creations of *evil* getters (getMonth(), getDay(), getYear()) for otherwise private details. Although it isn't obvious, it also tampers on the responsibilities of BorrowableItem by deciding by itself how penalty is calculated for each realization.

The moment you ask an object what it's exact type is, you should consider refactoring your code since introspective checks like instanceof, typeOf, etc. are symptoms of smelly design. You can think of these checks as analogies to racial discrimination since your client object asks the race (type) of the dependency.

Assigning the correct responsibilities

When you're introducing new behavior or information to a system, you should ask first: *who should be responsible for this behavior or information?*

- Who should be responsible for calculating the differences between dates? **Date should be responsible, not LibraryCard.**

- Who should be responsible for calculating the penalties of specific `BorrowableItem` realizations? **`BorrowableItem` realizations should be responsible, not `LibraryCard`**

The best implementation of the system contains multiple examples of SRP. The `Date` class should be responsible for subtracting dates and adding dates, not the `LibraryCard` class. Forcing `LibraryCard` to contain code for `Date` operations will force you into breaking the boundaries of your classes (e.g. creating getters to expose private attributes). `LibraryCard` should not be aware of **how** dates are subtracted to be able to subtract dates. The same is true for a `BorrowableItem`'s due date. The `LibraryCard` shouldn't contain the specifications as to how `BorrowableItems` calculate their due date. In fact `LibraryCard` shouldn't even be aware of the exact type of the `BorrowableItem` (instance violates this).

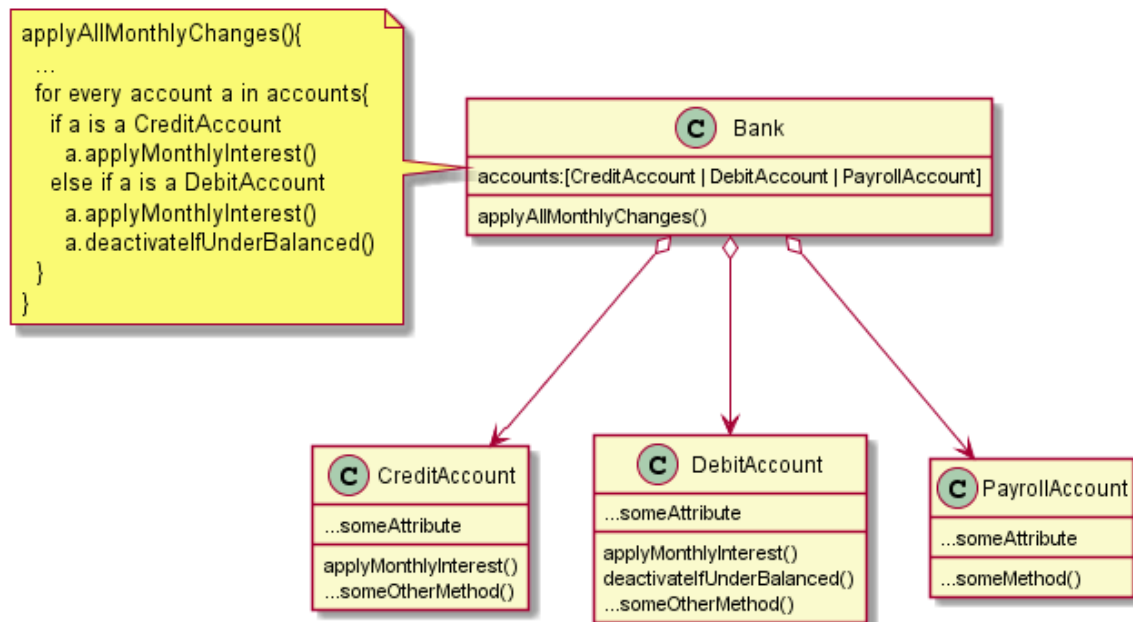
The process of designing these cohesive systems requires not only OOP design techniques but also domain knowledge. The designer of the system, should know how a library system operates so that, he/she can accurately simulate their responsibilities on code.

You can also apply SRP on individual methods inside objects. Methods should be responsible for one thing only. Keeping methods pure like these will help reduce unwanted side effects. The builder/manipulator naming scheme will help you with this. Also, a method should not be responsible for handling the problems such as errors/exceptions. The method should delegate that responsibility to the clients of that method. The methods itself should only report/raise/throw the problem. The best way to do this in OOP is by raising an exception.

Open/Closed Principle

A good class in OOP is both open and closed. It is open for extension but closed for modification.

To understand this principle let's have an example of a system that is closed for extension:



22

This is a system that indeed works perfectly. The bank will be able to apply the appropriate changes to the account types because of the if-else block that segregates the accounts based on its type (another instance of type discrimination so that's a hint that this is bad design). The problem with this is that whenever there are changes regarding the monthly changes or interest calculations, you would have to tamper with the contents of bank. This is **opening Bank up for modification**. Every time there's a change related to monthly updates you would have to do some kind of surgical procedure on Bank and rearrange its internal organs so that the change may be supported. This is extra rough on Bank because Bank shouldn't even be responsible for these behaviors (a violation of SRP). If there are new types of account, then you would have to open up Bank again and to add another else if block. Poor Bank, who knows how many more new types of accounts there are in the future.

Instead of rearranging the organs of your classes to accommodate changes to behavior they are not even responsible for, you should close the classes for modification and open them for extension instead:

²²Forgive the long Java-like method names, they're named as descriptive as possible so that I can skip actually explaining what they do.

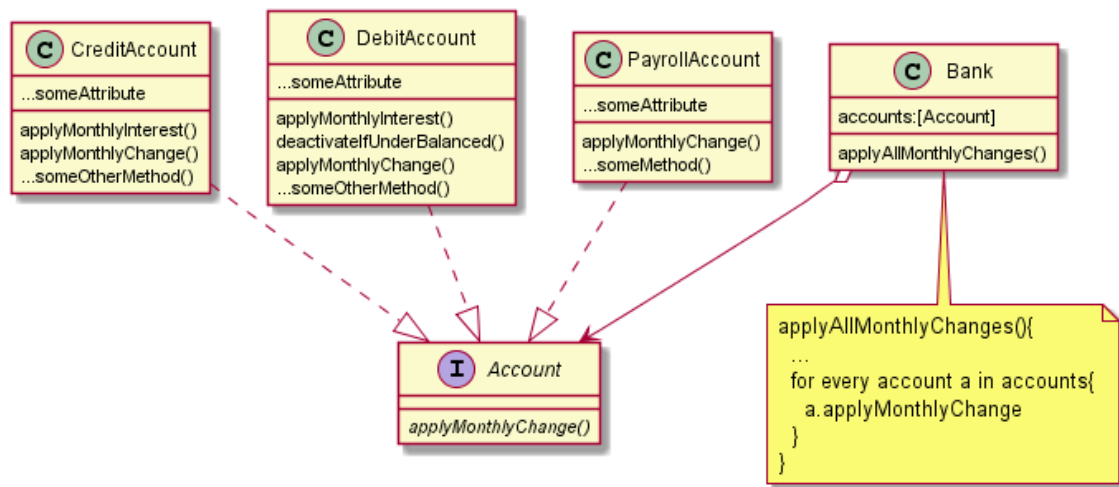


Figure 10: open for extension

Since `applyMonthlyChange()` is an abstract method of `Account`, all its realizations are required to implement it. We extend the functionality of `CreditAccount`, `DebitAccount`, and `PayrollAccount` by adding an extra method.

- Inside `CreditAccount.applyMonthlyChange()` you just call `applyMonthlyInterest()`,
- Inside `DebitAccount.applyMonthlyChange()` you call both `applyMonthlyInterest()` and `deactivateIfUnderBalanced()`
- Inside `PayrollAccount.applyMonthlyChange()` you do nothing

Is a method that does nothing inelegant? Not really because this is an accurate representation as to how a `PayrollAccount` changes every month— it doesn't change. Also, in the future, the inertness of `PayrollAccount` may change so at least you have the function prepared.

The previous library card example and bank examples are indeed similar. This is because introspective checks like `isinstance` are again symptoms of inelegant design.

Another example of this is how you need to augment the `PayrollAccount` class to contain the `recieveFund(float)` or `deposit(float)` method so that it can become a recipient for transfer.

Closed for modification does not mean you cannot change literally anything in the class. Of course if there are mistakes in the specific behavior of the class then you have to modify it.

Liskov-Substitution Principle

This principle basically dictates when should an object be a subtype or a realization of another object. Should `PayrollAccount` be a realization of `Account`? If you can substitute any an instance of `Account` with a `PayrollAccount` then the answer is yes. The same is true if you want to establish an inheritance relationship between `Account` and `PayrollAccount`. LSP is important because it ensures the polymorphic capabilities of your realizations and subtypes.

Interface Segregation Principle

Sometimes the subtypes/realizations of a certain object may have diverse functionality. Some subtypes can deposit, some subtypes can't, all subtypes can be recipient for transfers but not all can be senders. The diversity of functionality support may sometimes force the designer to pollute the system with methods that the subtypes don't actually use. PayrollAccount will not use deposit but since it realizes Account then we reluctantly add it. This is a violation of LSP which states that an object should not be forced to implement methods it doesn't use.

Role Interfaces

The best way to design these diverse systems is by refactoring your architecture to have diverse role interfaces instead.

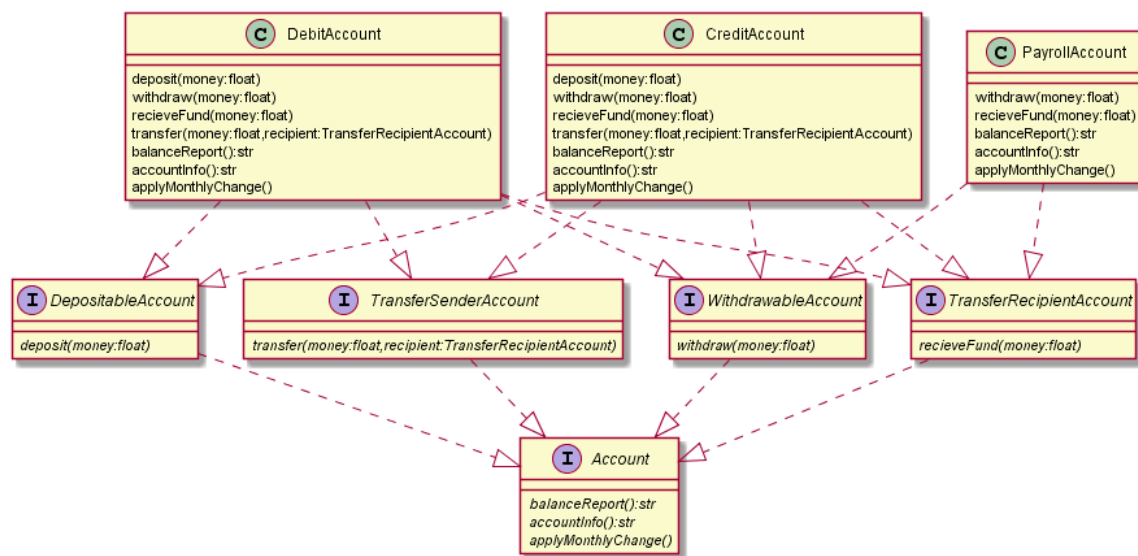


Figure 11: God class library card

Now instead of cluttering your realizations with useless methods, your system is now cluttered with role interfaces. This is a benevolent kind of clutter. Because more objects and looser dependencies make for a maintainable and therefore elegant system (in the same way a language with more words have less chance for ambiguity). Because of this clutter, you can have rich polymorphism without sacrificing ISP. Although be careful not to overdo it though. You wouldn't want a role interface for every conceivable method out there.

When you have role interfaces, you can refine lines of your code by describing which exact role interface applies. For example, instead of writing transfer as some Account to Account process they, it would be more specific to write it as a TransferSenderAccount to TransferRecipientAccount process.

Dependency Inversion Principle

The relationships between objects should be defined by surface instead of their interior. This means that How a bank interacts with an account should not be dependent on the implementation. The way Bank prints the balance report should not be by directly concatenating "Your balance

is" + account.balance. Bank should interact with an account by printing the return value the abstract builder method, `balanceReport()`. The class Bank should not dissect Account to retrieve the balance. It should tell the Account to build a balance report.

Optional Reading

Bailey D., (2009) SOLID Development Principles – In Motivational Pictures. Accessed August 31, 2020

Class Relationships

Introduction

The interactions between one an instance of a class to another is largely characterized by the relationship between them. Here we talk about relationships that define the polymorphism between classes and relationships that define dependency between objects.

Learning Outcomes

1. Differentiate realization relationships and specialization relationships
2. Describe how class abstract methods work in realization relationships
3. Differentiate aggregation relationships and composition relationships

Type Based Relationships

Type based relationships are characterized by how two classes are related to each other through ontological hierarchy. A mammal class's relationship with an animal class's relationship is type based. That is because a mammal is considered as an animal while an animal is not necessarily a mammal.

There are two type based relationships (there can be an extra one which is a type relationship that is sort of a hybrid of the two).

Realization

A realization relationship is a one way relationship that describes how something abstract is **REAL**ized by something concrete. Given a Realization class and some Abstraction class, The Realization class realizes the Abstraction class.

A realization relationship is also called an **implementation** relationship. An implementation, implements some interface. **Interface** is also a fitting term for abstractions because it is through these classes that other objects interact with each other.

An Abstraction is a special type of class that does not contain any implementation. This means that Abstraction doesn't have code that controls the form and behavior of the class. It only contains code that specify how this class interacts with other objects. This means that abstract classes only contain method names and type signatures with empty bodies.

These Abstraction classes appear useless at first since it doesn't do anything at all. In fact, you cannot even create an instance of an abstraction. Even if you do, it will be pointless since it doesn't have code that controls how it behaves.

An abstraction can only be useful if some other class realizes this abstraction. These Realization classes provide abstractions their form and behavior.

What's the point in maintaining some realization relationship between classes? If abstractions can only be used through their realizations, then why create the abstraction at all?

The importance of this seemingly pointless relationship lies in OOP's data hiding principle. We will explore more about why these relationships are very common in a future discussion about SOLID principles. For now, I'll show one of the reasons why this is useful through an example:

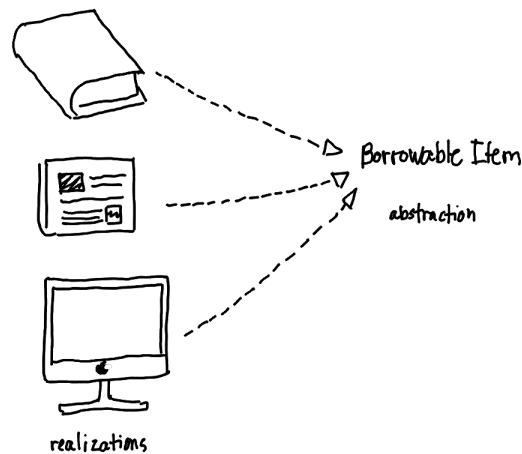


Figure 12: Realization Relationship Example

Consider a library system. In a library, you are able to borrow resources such as books, newspapers, and computers. When the library system interacts with these resources to facilitate transactions such as borrowing and returning, the library system treats these resource like general **Borrowable Items**. It doesn't really need to concern itself of the specific type. Operating like this is better for the library system for reasons such as maintainability and future proofing (this will be explained in detail in when we discuss SOLID design principles). Therefore, the best architecture to use in this situation is to make each resource type a realization of Borrowable Item. A BorrowableItem class would merely be an abstraction. This class would contain no behavior or form, it only contains specifications of how to interact with it. And it does make sense, I mean, how exactly does a general borrowable item behave or look like? It's abstract. What we know is that any BorrowableItem can be borrowed or returned, so we write empty `borrow()` and `return()` functions (it specifies how it interacts with others, but it doesn't have any specific behavior, these methods are called abstract methods).

Any realization of `BorrowableItem`, such as `Book` or `Newspaper` will be forced to implement the `borrow()` and `return()` methods as well. In general, all realizations are forced to implement the methods of its abstraction²³. This means, any realization needs to include these methods with each of their own method bodies for borrowing and returning. If books and newspapers are borrowed or returned in different manners, then you write different method bodies for each.

This is how realization relationships enable **polymorphism**, a `Book` is a `BorrowableItem`, allowing the library system to interact with it like any `BorrowableItem`. But at the same time `Book` is a book, so it behaves in the manner a book behaves.

By building all of these relationships, the library system is able to interact with resources without explicitly knowing which exact resource it is. The system knows that it is borrowing some instance of a borrowable item, but it does not know if it is a book or a newspaper. The exact type of this instance will then behave depending on its type. Although all this effort may seem unnecessary, you will learn in this course that through the establishment of these relationships, OOP is able to uphold one of its core design principles, **data-hiding**.

Abstractions can also realize abstractions. For example given an abstraction A, another abstraction, called B, can realize A. A class C then realizes B. When this happens, B does not need to implement A's abstract methods because B is an abstraction itself. Therefore, C will inherit all of A and B's abstract methods. The abstract methods of an abstraction cascades down to the realization realizing any of its realizations as well.

In this case C instances can be treated as B instances or A instances, but it will behave in the way C behaves.

Specialization

Specialization relationships are very similar to realization relationships. You can think of these specialization relationships as realizations but between two real/concrete classes. A specialized class specializes some general class. By establishing this relationship, you are able to extend the form and behavior of a specific class.

Specialization has plenty of names. This relationship is also called **extension** between the special/child/subclass and general/parent/super class. Another name for it would be **inheritance**.

²³While realizations are forced to implement all of its corresponding Abstraction's methods, realizations can have the option to implement extra methods that are not present in its abstractions.

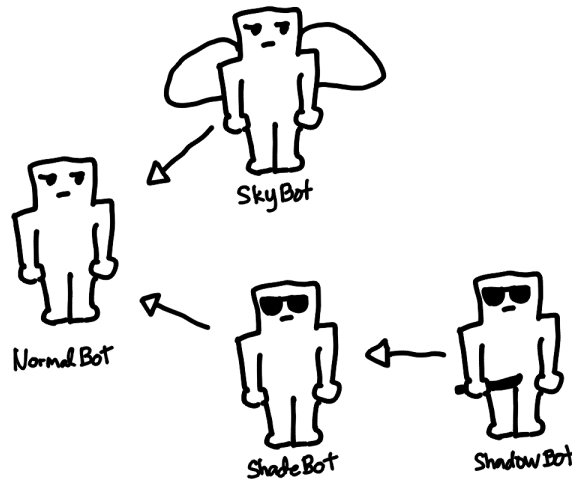


Figure 13: Specialization Example

Here's an example that would illustrate what the specialization relationship means. Consider a factory that builds robots. This factory is able to build `NormalBot`s which are the general type of robots. Instead of creating entirely separate mechanisms to build each type of robot, the factory is able to exploit the fact that other robots are just specializations of `NormalBot`. `SkyBot` is just the same as `NormalBot`, but it has extra flight capabilities. `ShadeBot` is just the same as `NormalBot`, but it has UV Protection. Because of this the factory is able to use the building recipes for `NormalBot` to build `SkyBot` and `ShadeBot`. All they need to do is to add some extra layers of construction such as adding wings or outfitting shades.

Specialization relationships work like this as well. When you write code for the general class, you do not need to rewrite it for specializations. What you write inside specializations are the attributes and method that make it special. If this specialization has flight capabilities then add attributes for wings and methods for flight. If this specialization behaves differently for some specific method then you only change that specific method.

Because of this relationship, you only need to write one copy of the code that is common for the general class and its specializations. This means that you do not need to rewrite said code, saving time and effort but more importantly, having one copy of code helps for maintainability. When the recipe of all robot types need to change, the factory only needs to change the recipe of `NormalBot`, all of special robots' recipes will change as well since they all use `NormalBot`'s recipe. When the code for the general class and the special classes need to be updated, changing the shared code found inside the general class will automatically affect special classes. Through this mechanism, specialization relationships enable **inheritance**, one of OOP's core design principle.

This is where the terms **inheritance** and extension make sense. Special class inherit all the attributes and methods of the general class. Special classes can also be thought of as extensions of the general class, since these special classes extend or tweak the capabilities of the general class.

You can also specialize, specializations. This is illustrated by ShadowBot, which is a special ShadeBot that has knife. Since ShadowBot is a special ShadeBot and ShadeBot is a special NormalBot, ShadowBot is automatically a specialization of NormalBot as well.

Specializations also allow polymorphism in the same way realizations do. A ShadowBot can be interacted with like any NormalBot or ShadeBot but since it is also a ShadowBot it will behave specifically like a ShadowBot.

Abstract Classes

An abstract class is something in between an abstraction and a generalization. It contains attributes and methods with bodies, but it also contains abstract methods as well. When classes specialize/realize abstract classes, they inherit the attributes and methods with bodies, but they are forced to implement the abstract methods as well. These relationships are sometimes used if the system requires a mix of inheritance and implementation between classes.

Multiple Type Relationships

It is possible for a class to realize/specialize multiple abstractions/generalizations. For example, given abstractions, A, B and C, and generalizations E, F, and G, a class X can realize/specialize all of them. When you do this, X will be forced to inherit all the abstract methods in A, B and C, and automatically inherit everything from E, F, and G.

Dependency Relationships

Dependency relationships, characterize how two classes interact with each other. A class which is dependent on another class, needs to know how to interact with it. These interactions range from being used as method parameters, being returned in methods, being used inside method bodies, being used as attributes, etc. A dependency relationship is one way (but it is also possible for two objects to be dependent on each other). A **client** class is dependent on some **dependency**.

Weak Dependency

On weak dependency relationships a client class merely **uses** instances of a dependency. This is when, a class uses other classes as method parameters, local variables, or return types.

Structural Dependencies

Structural dependencies, define relationships between a collective and its parts. If an object instance contains instances from a different class. Then they have a structural dependency.

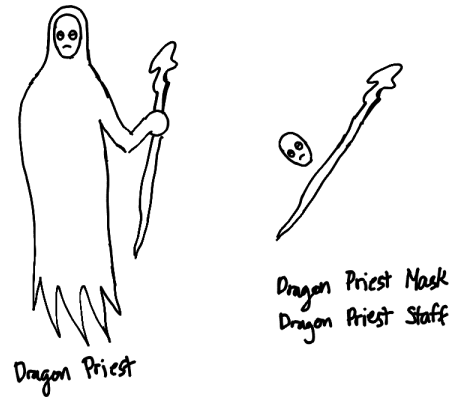


Figure 14: Dependency Example

Aggregation

When a dependency is an aggregate of some client, it means that the client merely **has** instances of this dependency. These relationships are the looser forms of structural dependency, because the dependency instance can exist outside the lifetime of the client instance.

For example, in a videogame, a hostile enemy instance of `DragonPriest` spawns equipped with instances of `DragonPriestMask` and `DragonPriestStaff`. The `DragonPriest` instance is a client of the dependencies `DragonPriestMask` and `DragonPriestStaff`. `DragonPriest` interacts with these dependencies to calculate its attack damage, its defenses, etc. But when this specific instance of `DragonPriest` is defeated, it despawns, leaving behind its `DragonPriestMask` and `DragonPriestStaff`. These instances will continue to exist since it can still be used by other client objects in the game, such as the player character, some storage chest or whatever. This means that the relationship between `DragonPriest` and the dependencies `DragonPriestMask` and `DragonPriestStaff` is aggregation.

Composition

Composition relationships are ownership dependencies. When a client is composed of some dependency, this means that the client **owns** the instances of this dependency. These relationships are stronger forms of dependency since the existence of the dependency instance is tied to the client, meaning, the dependency ceases to exist outside the lifetime of the client instance.

In the same videogame, the hostile enemy instance of `DragonPriest` appears in game using some `DragonPriestCharacterModel` instance. When the `DragonPriest` instance is defeated, it despawns. It makes no sense for the `DragonPriestCharacterModel` instance to stay behind after

the DragonPriest instance is defeated, therefore it ceases to exist as well. This means that the relationship between DragonPriest and the dependency DragonPriestCharacterModel is composition.

Blackburn, Patrick, Johan Bos, and Kristina Striegnitz. 2006. *Learn Prolog Now!* Texts in Computing 7. College Publications.

Church, Alonzo. 1932. "A Set of Postulates for the Foundation of Logic." *The Annals of Mathematics* 33 (2): 346. <https://doi.org/10.2307/1968337>.

Cote, Joe. 2025. "What Is Computer Programming." In *Southern New Hampshire University*.

Hosch, William. 2010. "Peano Axioms." In *Encyclopaedia Britannica*. Encyclopaedia Britannica, Inc. <https://www.britannica.com/science/Peano-axioms>.

MovGP0. 2013. *Overview of the Various Programming Paradigms According to Peter Van Roy*.

PARADIGM Definition & Meaning Dictionary.com. n.d. Accessed August 10, 2026. <https://www.dictionary.com/browse/paradigm>.

Steele, Guy L., and Richard P. Gabriel. 1996. "The Evolution of Lisp." In *History of Programming Languages—II*, edited by Thomas J. Bergin and Richard G. Gibson. ACM. <https://doi.org/10.1145/234286.1057818>.

Triska, Markus. 2026. *The Power of Prolog*. Metalevel.at.